



Universidade Estadual de Campinas
Faculdade de Engenharia Civil
IC907-Inteligência Artificial Aplicada a Problemas
de Engenharia

**Projeto 2 - Redes Neurais do Zero utilizando
técnica de *dropout***

Gabriel Silva 235038
Luis Felipe Oliveira Santos 299675
Yan de Miranda Zimmermann Lobo 195649

Campinas/SP
2025

1. Introdução

Redes neurais artificiais (ANN do inglês *Artificial Neural Networks*) são um tipo de aprendizagem de máquina, isto é, de maneira simplificada, um método para qual seja possível que máquinas por meio da exposição a diferentes dados aprendam e realizem tarefas de maneira autônoma. A concepção teórica de uma rede neural é inspirada no funcionamento do neurônio humano, ou seja, para que ocorra o processamento de um dado de entrada há vários neurônios, ativações e interconectividades que resultam em uma informação processada. Essa interconectividade entre neurônios é um dos grandes diferenciais das ANNs, pois permite, por exemplo, o aprendizado considerando possíveis interdependência entre dados. Uma ANN é formada, simplificada, por diferentes camadas, sendo uma primeira camada relativa a dados de entrada e a última aos resultados relativos aos dados de entrada. As camadas intermediárias são chamadas de camadas densas (encontrado na literatura em inglês como *Dense Layers*), elas são compostas por diversos neurônios que são interconectados entre essas camadas, isto é, cada neurônio de uma camada densa é conectado com cada neurônio da próxima camada densa. O aprendizado da ANN é realizado por meio do ajuste de pesos e constantes (*weights* e *biases*), sendo esses pesos ajustados para cada conexão entre neurônios e as constantes para cada neurônio. Nesse sentido, as ANNs podem ser interpretadas como uma sequência de transformações lineares, cujo resultado passa por uma função de ativação, o que permite as ANNs generalizarem uma grande variedade de problemas. Entretanto, muitas vezes o treinamento das redes não resulta em uma generalização, mas em uma adaptação aos dados de treinamento, como uma memorização, conhecido como *overfitting*. Deste modo, métodos de regularização são utilizados para evitar que ocorra o *overfitting*, um desses métodos é conhecido como *Dropout*, que consiste em aleatoriamente desativar uma parcela de neurônios e suas conexões durante o treinamento da rede, de modo a forçar um treinamento que resulte em uma representação mais robusta do problema. Diante disso, este trabalho tem como objetivo implementar o algoritmo do método de regularização *Dropout* e comparar com uma implementação base, apresentando uma comparação entre as curvas de perda, o custo computacional e a sensibilidade dos hiperparâmetros envolvidos.

2. Dropout

O Dropout pode ser definido como um método de regularização de uma rede neural, ou seja, é uma técnica que pode ser utilizada para melhorar a capacidade de generalização da ANN, reduzindo casos de overfitting. Outros exemplos de métodos que podem ser utilizados para o mesmo objetivo são as regularizações L1 e L2, onde os pesos são penalizados. O Dropout é incluído como uma camada fantasma na rede neural, na qual nessa camada alguns neurônios são desativados por época de treinamento, a fim de evitar com que a rede neural seja muito dependente de neurônios em específico e que neurônios de camadas posteriores fiquem muito dependentes de neurônios de camadas anteriores. A escolha dos neurônios que serão desativados, isto é, suas saídas serão zeradas, é feita aleatoriamente pelo método utilizando um hiperparâmetro de probabilidade [1, 2].

O dropout introduz um hiperparâmetro r (taxa de dropout) que representa a fração de saídas que serão zeradas durante cada passo de treinamento. Para uma entrada de ativação x na camada onde se aplica dropout, define-se uma máscara binária amostrada de uma distribuição Bernoulli com parâmetro $1-r$. A ativação com dropout invertido (convenção usada pela maioria dos frameworks modernos) é

$$m \sim \text{Bernoulli}(1 - r)$$

$$\overline{x} = \frac{x * m}{1 - r}$$

Dessa forma, a esperança das ativações durante treinamento permanece igual à ativação original, e a camada pode ser usada sem ajustes no modo de avaliação.

3. Metodologia

A metodologia adotada consistiu na construção gradual de uma arquitetura de rede neural totalmente programada em Python, estruturada em três módulos principais. O objetivo foi permitir controle total sobre cada componente da rede, possibilitando a análise detalhada do impacto do *dropout* sobre o processo de treinamento e a generalização do modelo.

3.1. Estruturação inicial — arquivo *BaseClasses*

O primeiro módulo desenvolvido, denominado *BaseClasses*, fornece a infraestrutura fundamental sobre a qual os demais componentes da rede neural foram implementados. Esse arquivo contém três classes principais:

- **Layer**: classe base que define a estrutura genérica para camadas da rede. Ela fornece métodos fundamentais como armazenamento de entradas, saídas e gradientes, servindo como a base para camadas específicas.
- **Loss (Layer)**: classe responsável pelo cálculo da função de perda, herdando da classe *Layer* para permitir integração direta com o fluxo de *forward* e *backward*.
- **Optimizer**: classe genérica para algoritmos de otimização, oferecendo o arcabouço para extensões como SGD com *momentum* e *learning rate decay*.

Esse primeiro módulo funciona como uma base abstrata que uniformiza a interface das outras camadas implementadas posteriormente, garantindo consistência entre componentes.

3.2. Implementação das funcionalidades da rede — arquivo *NNClasses*

O segundo módulo, denominado *NNClasses*, reúne as funcionalidades específicas de redes neurais artificiais, incluindo:

- Funções de ativação (ReLU, Sigmoid, Tanh e Leaky_ReLU)
- Classe *Layer_Dense*, responsável pela multiplicação de pesos e adição de vieses.
- Função de perda *MSE (Mean Squared Error)*.
- Implementação completa do otimizador *SGD*, incluindo as variantes com *momentum* e *decay_rate*.

Dentro desse arquivo encontra-se também a implementação da camada de dropout, construída manualmente.

3.3. Execução da rede neural — arquivo *NeuralNetwork*

A classe *Layer_Dropout* foi desenvolvida seguindo o funcionamento clássico do *dropout* aplicado somente durante o modo de treinamento. Ao ser inicializada, a camada recebe dois parâmetros:

- Modo: indica se o dropout deve ser aplicado (treinamento) ou ignorado (inferência).
- Probabilidade de dropout: probabilidade de desligamento dos neurônios, garantindo que $0 \leq p \leq 1$

Durante o *forward pass* em modo de treinamento, a camada:

1. Gera uma máscara binária por meio de uma distribuição `np.random.binomial`, mantendo cada neurônio com probabilidade $1 - p$.
2. Aplica o esquema de *inverted dropout*, dividindo a máscara por $(1 - p)$. Isso garante que a esperança da ativação seja mantida, ou seja,

$$E[\bar{x}] = x,$$

evitando mudanças de escala entre treinamento e inferência.

3. Multiplica a máscara pelas ativações de entrada para obter a saída efetiva da camada.

```

class Layer_Dropout(Layer):
    def __init__(self, is_training_mode: bool = False, dropout_prob: float = None):
        super().__init__()

        self.is_training_mode: bool = is_training_mode

        self.dropout_prob: float = dropout_prob if dropout_prob is not None else 0.4

        if not 0.0 <= self.dropout_prob <= 1.0:
            raise ValueError("Dropout probability must be between 0 and 1.")

        self.dropout_mask: float = None

    def forward(self, inputs: np.ndarray):
        if self.is_training_mode:

            self.dropout_mask = np.random.binomial(1, (1-self.dropout_prob), size=inputs.shape) / (1-self.dropout_prob)
            self.inputs = inputs #* Save inputs for backpropagation
            self.output = inputs * self.dropout_mask

        else:
            self.dropout_mask = np.ones_like(inputs)

            self.inputs = inputs #* Save inputs for backpropagation
            self.output = inputs

    # Backward pass
    def backward(self, dvalues: np.ndarray):
        self.dinputs = dvalues * self.dropout_mask

```

Figura 1 — Implementação da classe Dropout.

3.4. Geração da rede neural — arquivo *NeuralNetwork*

O módulo final, denominado *NeuralNetwork*, contém a classe que coordena todo o processo de treinamento. Essa classe:

- Organiza as camadas em sequência.
- Gerencia o *forward* e *backward pass* completos.
- Calcula a perda total e gera gráfico de perda por época.
- Atualiza os pesos usando o otimizador selecionado.

Com esse módulo, torna-se possível realizar testes experimentais, medir o overfitting e avaliar comportamento da rede em diferentes condições.

3.5. Estrutura dos testes

A etapa de testes teve como objetivo principal analisar de forma controlada o comportamento da rede neural em situações com e sem overfitting, avaliando em que condições o *dropout* se torna necessário e qual é seu impacto sobre o processo de treinamento. Para isso, os experimentos foram conduzidos de maneira progressiva,

criando cenários específicos que favoreciam a ocorrência de *overfitting* e, em seguida, cenários nos quais o fenômeno era mitigado.

A primeira etapa consistiu em projetar casos de teste que deliberadamente levassem a rede a super ajustar os dados . O *overfitting* foi incentivado pela combinação de fatores como:

- **Pequeno número de pontos de treinamento**, facilitando a memorização dos dados.
- **Uso de modelos altamente flexíveis**, como redes com número elevado de neurônios, capazes de se ajustar até mesmo a variações que não representam informação real.

Esses fatores foram explorados para gerar cenários controlados onde fosse possível observar, de maneira clara, a discrepância entre o erro de treinamento e o erro de validação, caracterizando o *overfitting*. Em paralelo, foram criados cenários onde o modelo tendia a não super ajustar, por exemplo usando menos neurônios, ajustando profundidade ou utilizando conjuntos de dados maiores e mais suaves.

3.6. Testes com ajustes de hiperparâmetros – *learning rate decay* e *momentum*

Antes de aplicar o *dropout*, foi conduzida uma etapa de experimentação envolvendo apenas o otimizador *SGD* e suas variantes implementadas no projeto. Nessa fase, avaliaram-se:

- mudanças no *decay rate*, responsável por reduzir gradualmente o *learning rate* ao longo das épocas,
- inclusão ou não do *momentum*, que acelera a convergência e suaviza oscilações.

Esses testes tiveram dois propósitos:

1. Verificar se o controle adequado do processo de otimização poderia, por si só, reduzir o aparecimento de *overfitting*, e
2. Estabelecer uma linha de base para comparação com os experimentos subsequentes envolvendo o *dropout*.

O uso de *learning rate decay* tende a estabilizar o treinamento, enquanto o *momentum* influencia a capacidade do modelo de escapar de mínimos locais. Entretanto, apesar de afetarem a dinâmica de aprendizado, esses mecanismos não são métodos de regularização.

3.7. Aplicação do Dropout

Após a etapa de testes envolvendo ajustes do otimizador, foi iniciada a fase dedicada exclusivamente à avaliação do *dropout* como método de regularização. Essa etapa consistiu em inserir camadas de *dropout* em diferentes regiões da rede neural e analisar como essa técnica alterava o comportamento estrutural do treinamento.

Foram testadas diferentes configurações de posicionamento das camadas de *dropout*, incluindo:

- aplicação apenas em uma camada,
- aplicação em mais de uma camada simultaneamente,
- aplicação em camadas próximas à entrada, próximas à saída ou distribuídas ao longo da arquitetura.

Essa abordagem permitiu observar como o efeito regularizador variava conforme a camada em que os neurônios eram desativados. Além do posicionamento, a taxa de desligamento dos neurônios também foi variada. Valores diferentes de probabilidade de *dropout* foram testados, como:

- taxas mais baixas (por exemplo, 0.1 a 0.2), que introduzem regularização leve,
- taxas médias (0.3 a 0.5), frequentemente utilizadas como padrão,
- taxas mais altas (acima de 0.5), que impõem maior aleatoriedade e reduzem significativamente a coadaptação entre neurônios.

O objetivo dessa etapa foi analisar como a intensidade da regularização afeta a estabilidade do aprendizado e a capacidade da rede de entender padrões dos dados sintéticos.

3.8. Função inicial

A primeira etapa da análise foi conduzida utilizando a função seno

$$y = \text{Sen}(\pi x)$$

amostrada em 25 pontos no intervalo $[-2,3]$. Esse conjunto serviu como base inicial para avaliar o comportamento da rede neural e observar situações de ajuste adequado e de *overfitting*. Ao longo do estudo, uma segunda função também foi introduzida com o objetivo de explorar padrões mais complexos.

Os experimentos foram organizados de forma sequencial, permitindo observar isoladamente o efeito de cada mecanismo de otimização ou regularização. Assim, os resultados apresentados a seguir seguem a ordem:

1. Treinamento utilizando apenas o otimizador básico (SGD).
2. Aplicação de *learning rate decay* para analisar sua influência na estabilidade da convergência.
3. Inclusão do termo de *momentum* no processo de otimização.
4. Aplicação do *dropout*, com variação das taxas e da posição das camadas em que foi inserido.

Para essa função inicial, a rede neural adotada foi definida com 100 neurônios na primeira camada densa e 50 neurônios na segunda, configuração obtida após alguns testes exploratórios e ajustes preliminares. O processo de treinamento utilizou *learning rate* igual a 0.1 e 500.000 épocas, de forma a permitir que qualquer tendência ao *overfitting* ou instabilidade ficasse evidente durante o treinamento.

Em todos os casos foram gerados gráficos de perda por época, razão entre perdas de teste e treino, além das curvas comparando a função real com a predição da rede neural.

3.9. Segunda Função

A segunda fase experimental utilizou como função alvo

$$y = x^2 - \text{Sen}(5x),$$

selecionada por combinar um termo polinomial suave com um componente oscilatório de maior frequência. Essa composição aumenta a complexidade do problema e demanda maior capacidade de representação da rede em comparação à função utilizada na etapa inicial.

Para essa fase, a arquitetura foi aprofundada, passando a conter três camadas ocultas com 50 neurônios cada. As funções de ativação foram configuradas de forma híbrida: Leaky ReLU na primeira e na última camada, e *Tanh* na camada intermediária. Essa escolha foi adotada porque, nos testes preliminares sem dropout, levou à obtenção de perda de teste igual a zero, caracterizando um caso de *overfitting* extremo. Justamente por isso, essa função e essa arquitetura foram mantidas como base para analisar, de forma mais clara, o impacto da aplicação do dropout em um cenário onde a rede tem alta capacidade de memorização.

4. Resultados e discussões

4.1. Função inicial

4.1.1. Resultados com Otimizador SGD (sem ajustes adicionais)

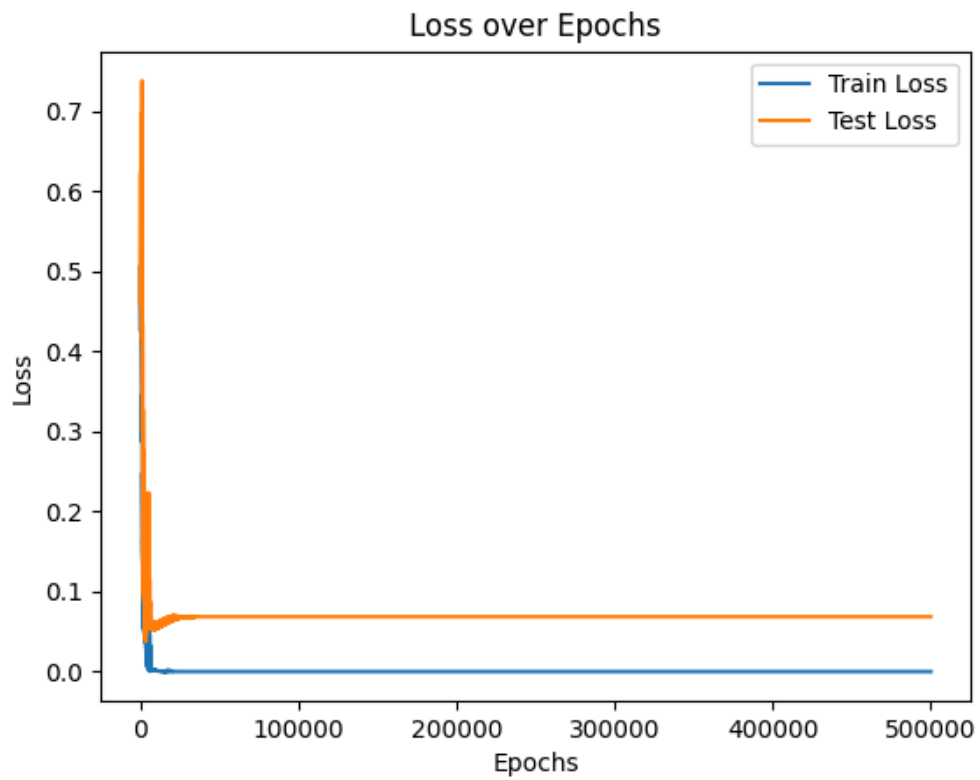


Figura 2 — Evolução da perda durante o treinamento.

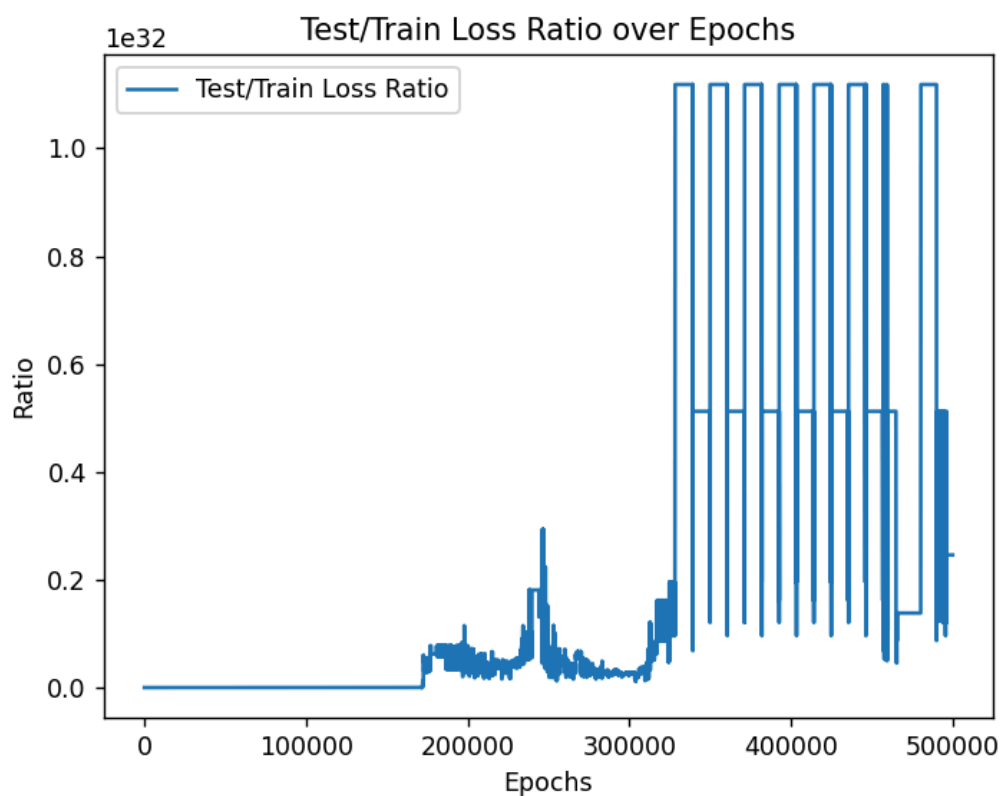


Figura 3 — Razão entre perdas de teste e treino ao longo do treinamento.

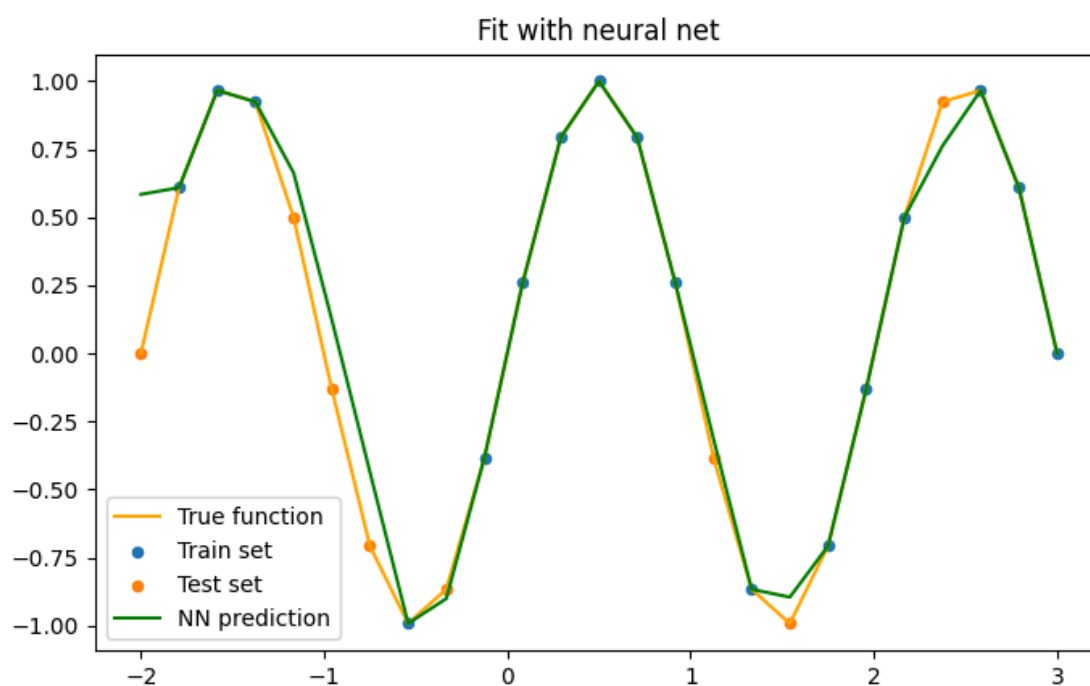


Figura 4 — Comparação entre a função alvo e a predição obtida pela rede neural.

Este primeiro experimento resultou em uma perda praticamente igual a zero, um indicativo direto de *overfitting*. Esse comportamento era esperado, dado que o número reduzido de pontos amostrados favorece a memorização da função pela rede, levando-a a ajustar-se quase perfeitamente aos dados disponíveis.

Antes de avançar para um cenário de “overfitting completo”, optou-se por expandir a análise explorando outros aspectos do processo de treinamento. Assim, aumentou-se o número de pontos para 50, para tornar a validação mais realista e permitir a avaliação do impacto de técnicas como *learning rate* adaptativo (*decay*) e momentum, além de avaliar efeitos do *dropout*. Em testes posteriores, uma segunda função também foi empregada; novamente observou-se perda praticamente nula, e esse caso foi utilizado especificamente para estudar o efeito isolado do dropout nesse cenário.

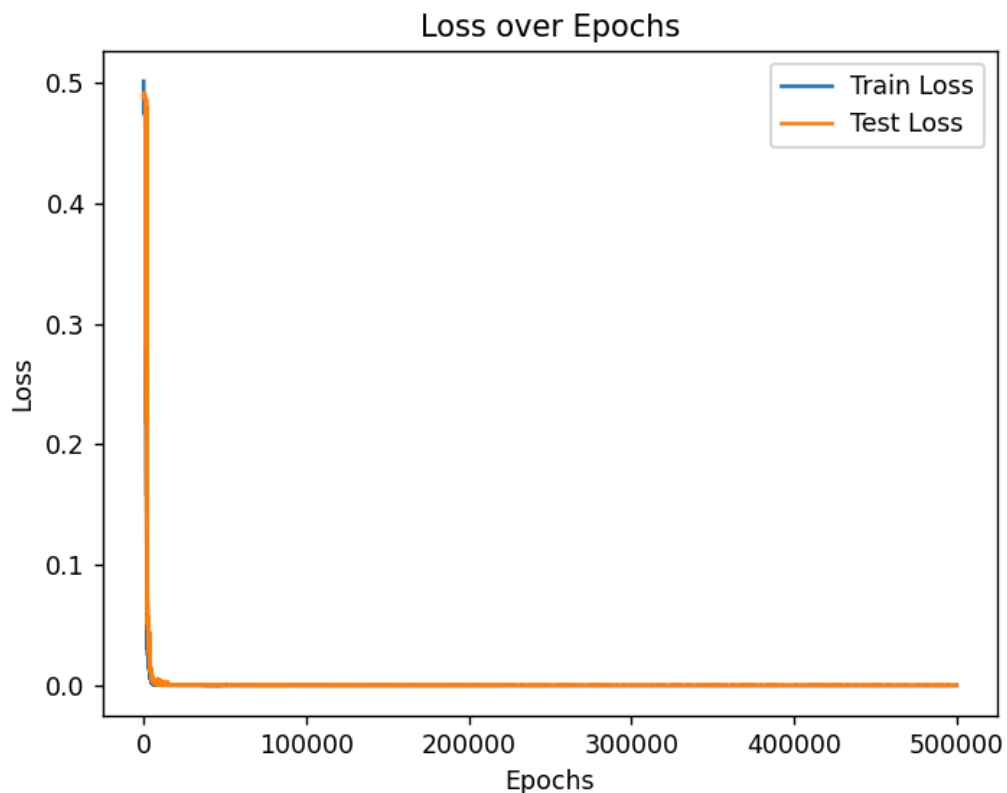


Figura 5 — Evolução da perda durante o treinamento.

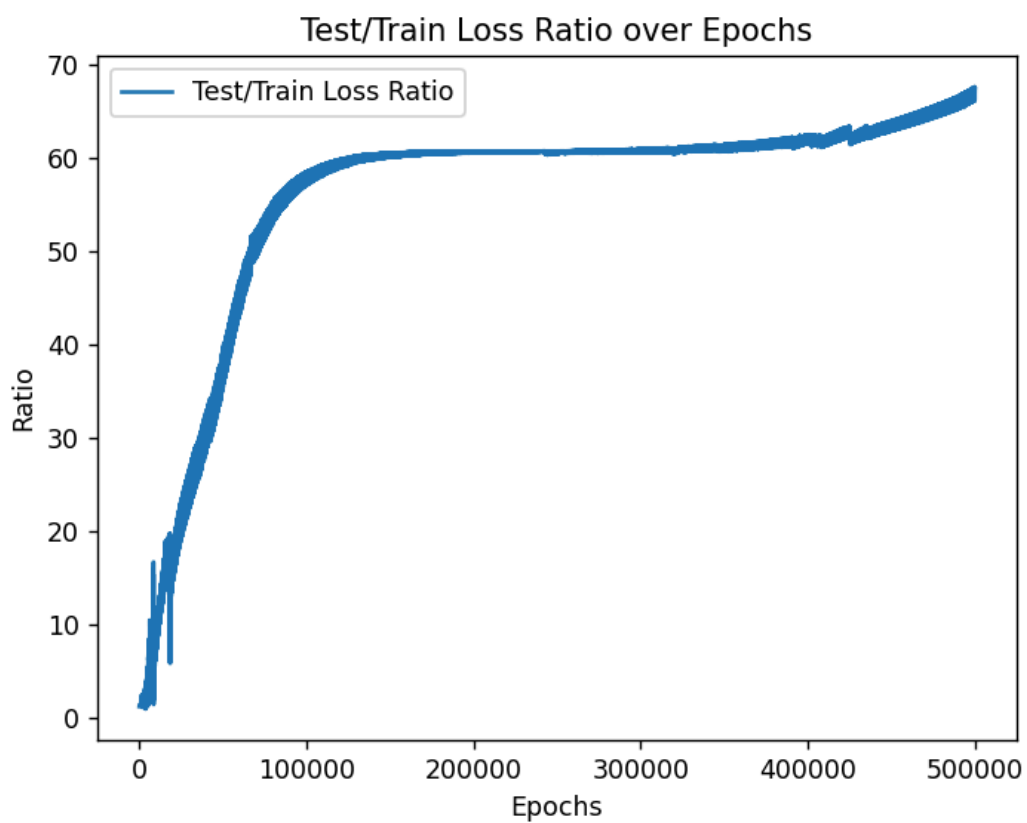


Figura 6 — Razão entre perdas de teste e treino ao longo do treinamento.

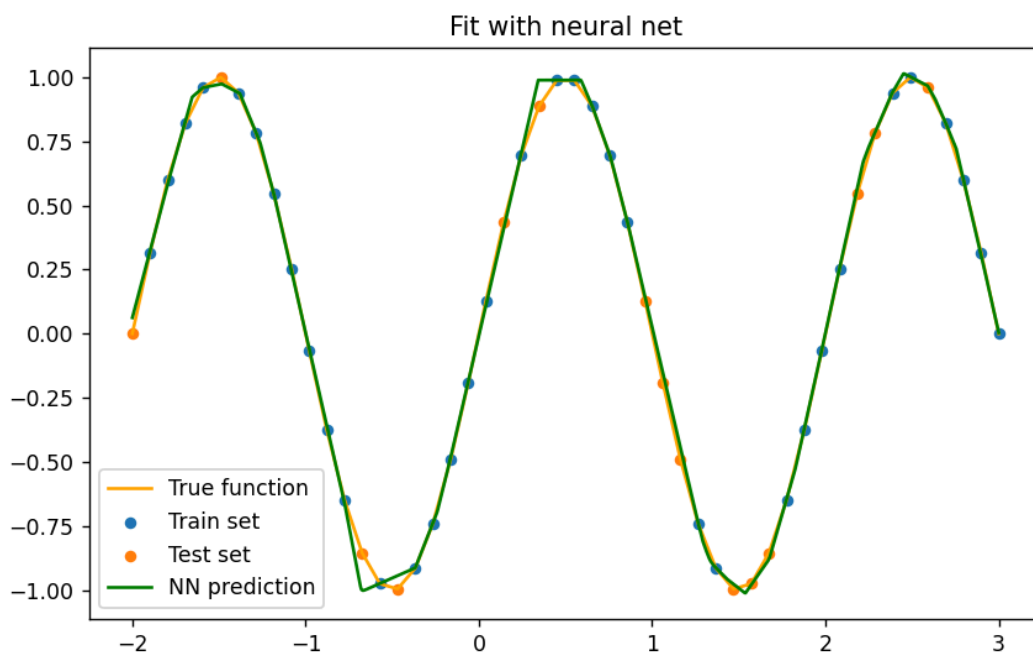


Figura 7 — Comparação entre a função alvo e a predição obtida pela rede neural.

4.1.2. Resultados com Learning Rate Decay

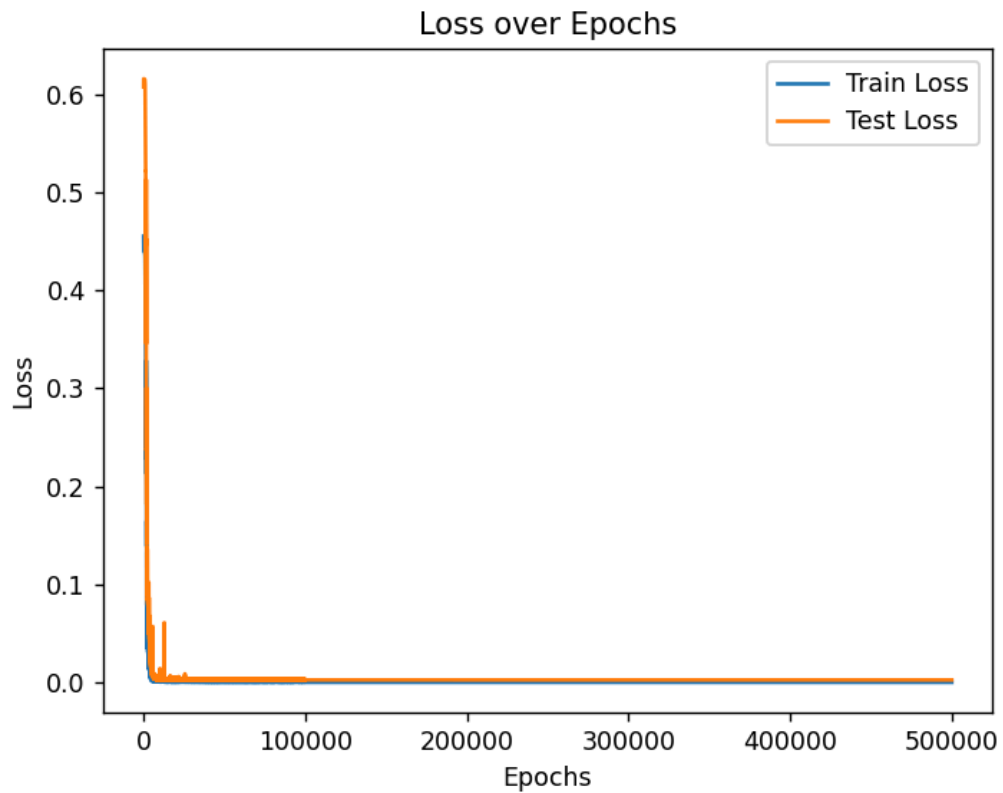


Figura 8 — Evolução da perda durante o treinamento.

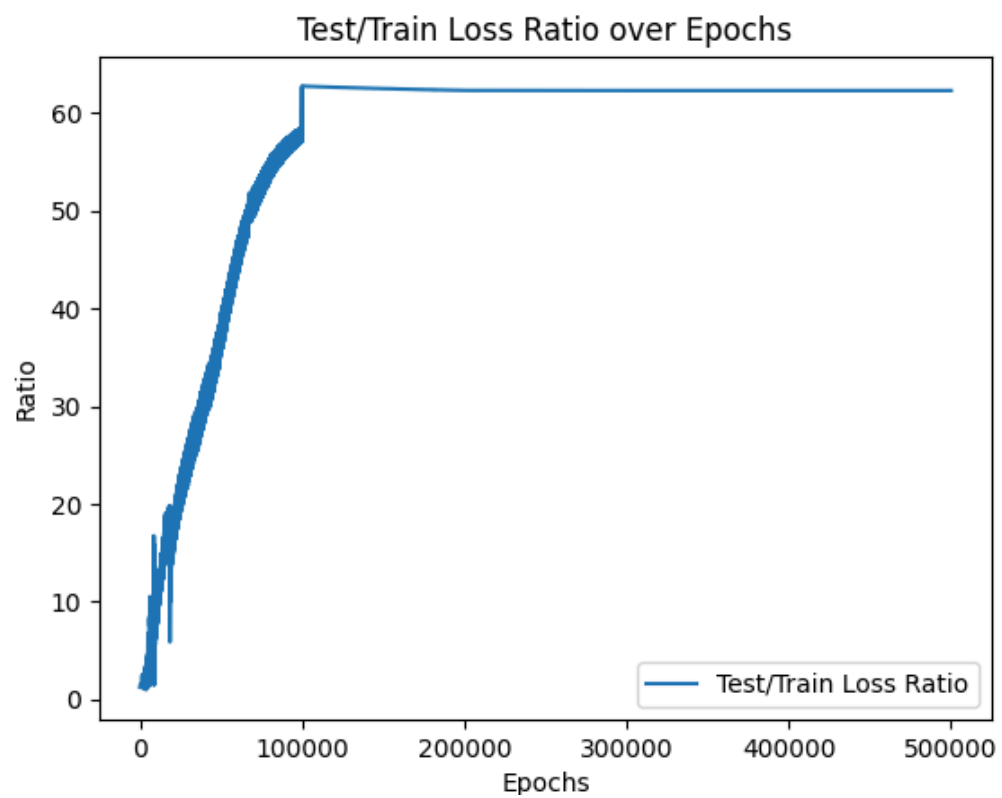


Figura 9 — Razão entre perdas de teste e treino ao longo do treinamento.

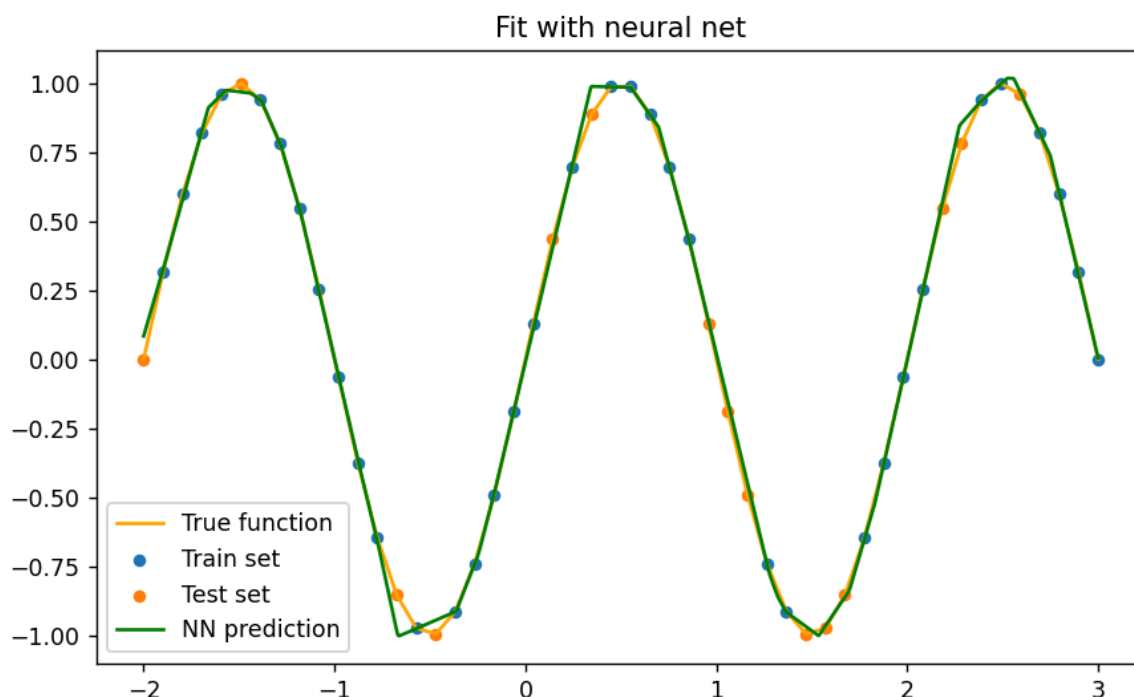


Figura 10 — Comparação entre a função alvo e a predição obtida pela rede neural.

A aplicação de *learning rate decay* não alterou de forma significativa o ajuste final da curva, ou seja, a predição da rede continuou praticamente idêntica ao caso base com SGD puro. No entanto, o uso do *learning rate decay* estabilizou a relação entre as perdas de teste e treino, fazendo com que seu crescimento atingisse um “platô”, indicando que a rede interrompeu seu processo de aprendizado após certo ponto. Como o decaimento reduz progressivamente a taxa de aprendizado, o valor do *learning rate* torna-se tão pequeno que as atualizações de peso passam a ser praticamente nulas.

4.1.3. Resultados com Momentum

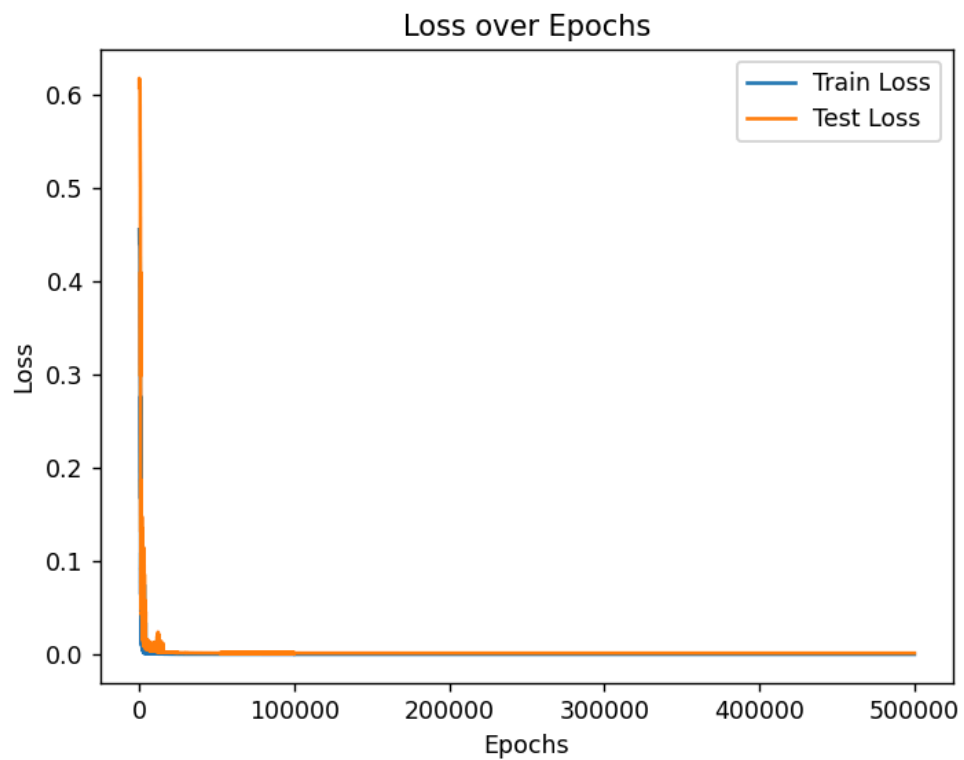


Figura 11 — Evolução da perda durante o treinamento.

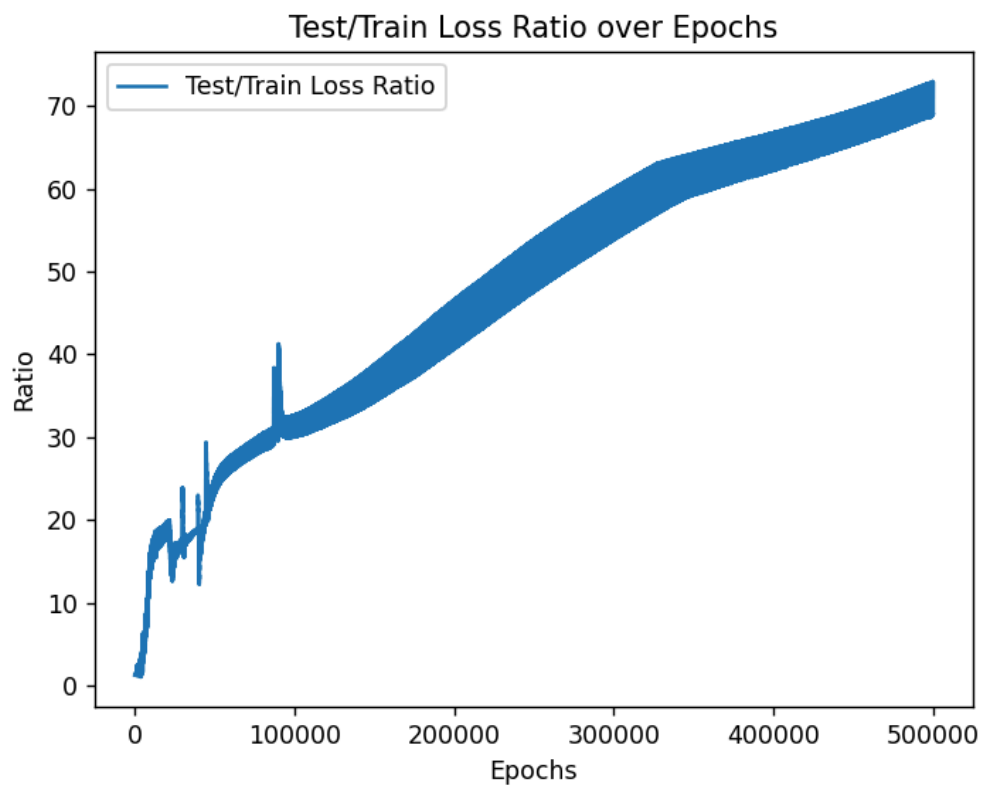


Figura 12 — Razão entre perdas de teste e treino ao longo do treinamento.

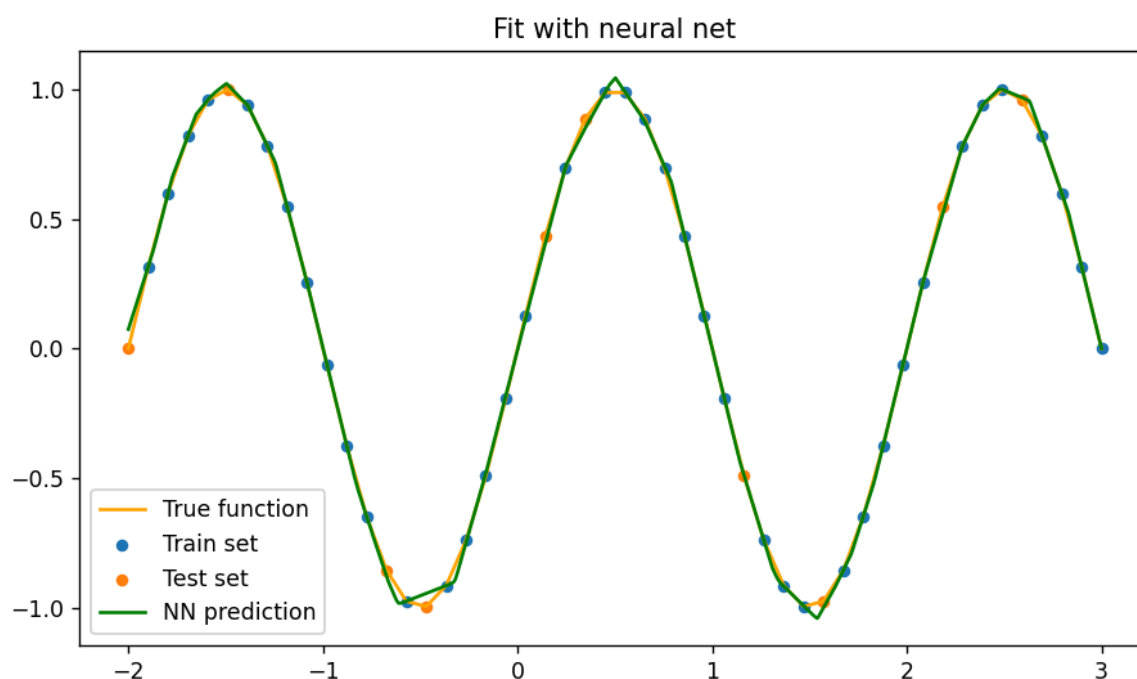


Figura 13 —Comparação entre a função alvo e a predição obtida pela rede neural.

A aplicação de *momentum* também não alterou o ajuste final da curva. A introdução dessa variável produziu o efeito de aumentar de forma mais acentuada a relação entre perda de teste e treino, sugerindo que o termo adicional no gradiente intensificou a tendência da rede de se ajustar mais fortemente ao conjunto de treinamento.

4.1.4. Resultados com Dropout

No primeiro teste, a técnica foi aplicada somente à primeira camada, utilizando uma taxa de 0.2. Em seguida, repetiu-se o procedimento aumentando a taxa para 0.4, permitindo observar o impacto de uma regularização mais intensa ainda restrita à primeira camada.

Posteriormente, os testes passaram a considerar o uso simultâneo do dropout nas duas camadas. Inicialmente, ambas receberam uma taxa de 0.2, e, por fim, foi avaliada a taxa de 0.4 nas duas camadas. Essa progressão permitiu analisar de forma controlada como a intensidade e a distribuição do dropout ao longo da rede influenciam o padrão de *overfitting* e o comportamento geral do treinamento.

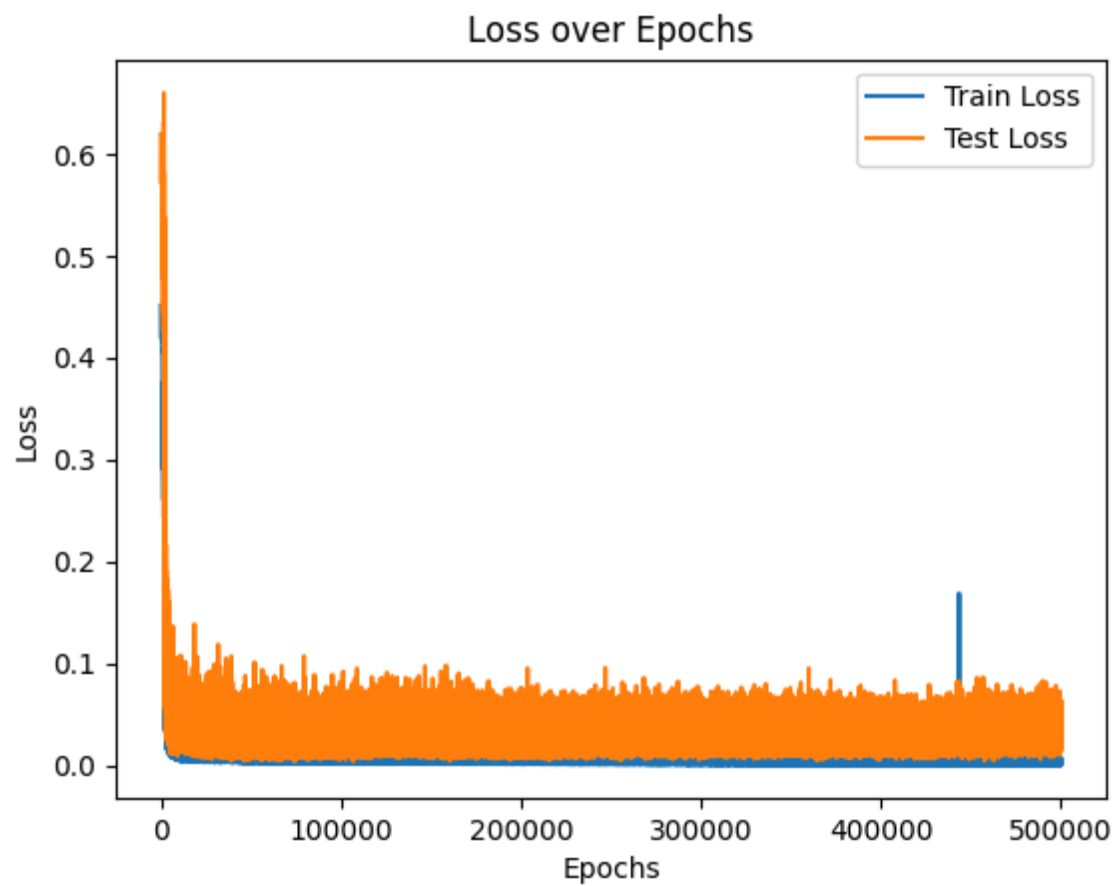


Figura 14 — Evolução da perda durante o treinamento - com probabilidade de dropout de 0.2 na primeira camada.

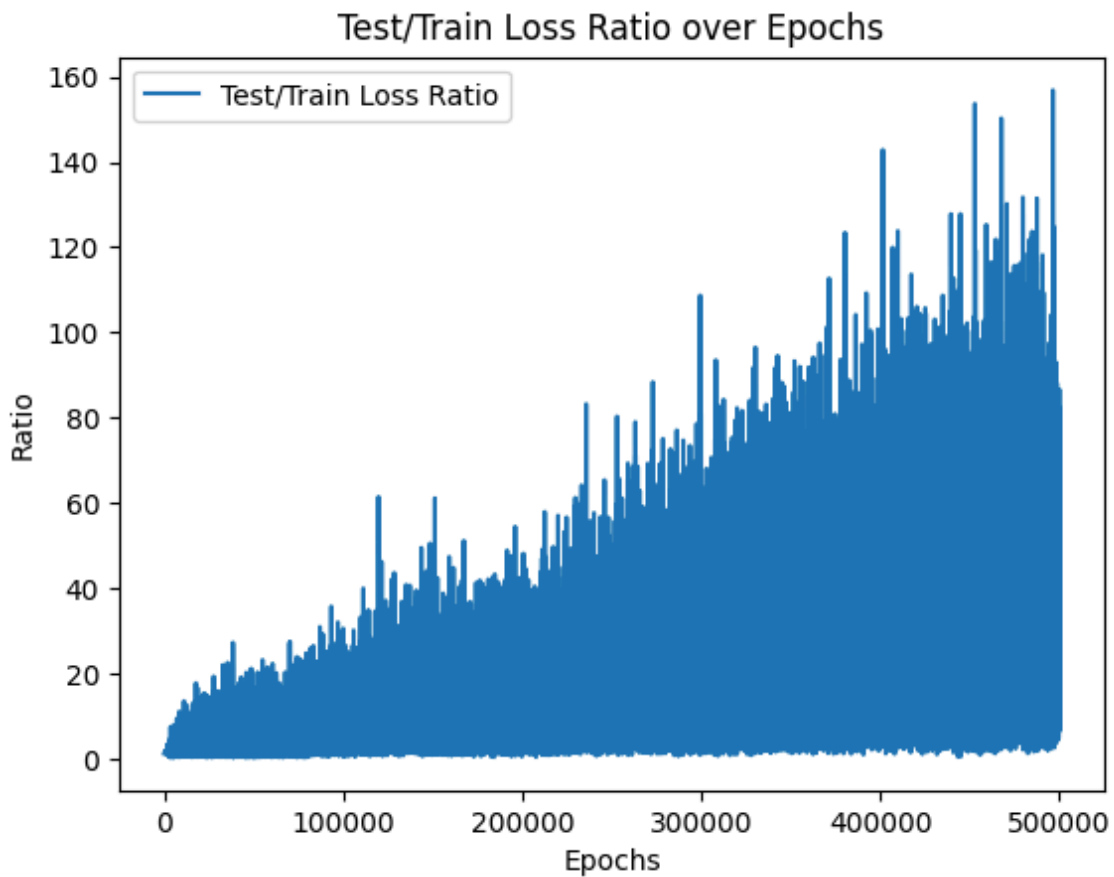


Figura 15 — Razão entre perdas de teste e treino ao longo do treinamento - com probabilidade de dropout de 0.2 na primeira camada.

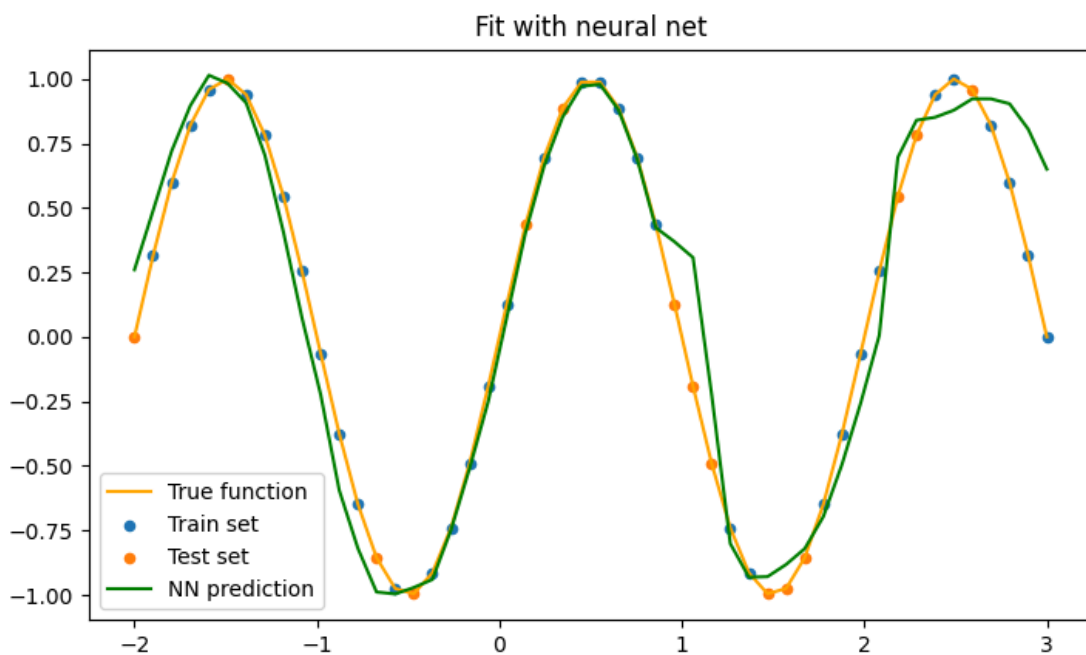


Figura 16 - Comparação entre a função alvo e a predição obtida pela rede neural.
- com probabilidade de dropout de 0.2 na primeira camada.

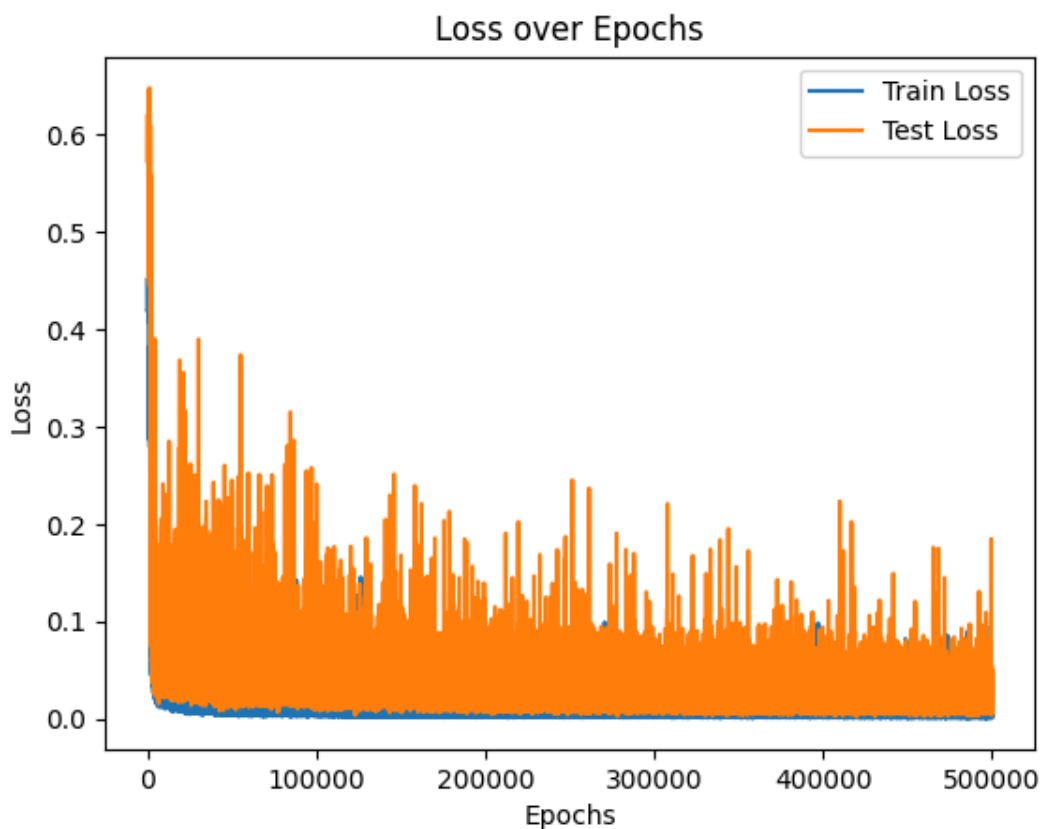


Figura 17 — Evolução da perda durante o treinamento - com probabilidade de dropout de 0.4 na primeira camada.

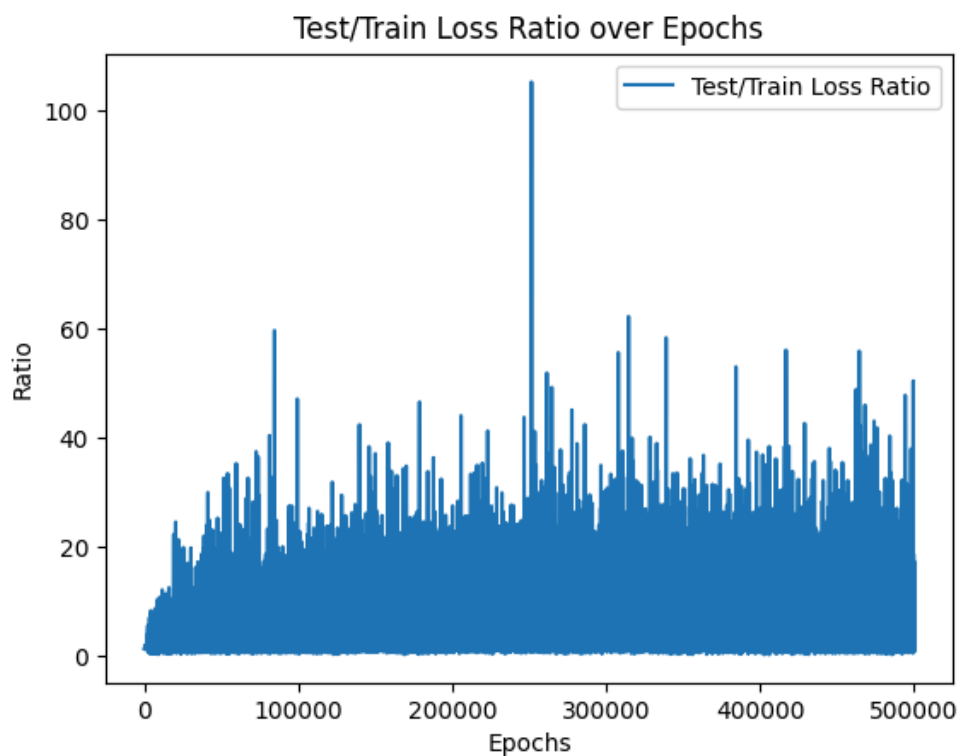


Figura 18 — Razão entre perdas de teste e treino ao longo do treinamento - com probabilidade de dropout de 0.4 na primeira camada..

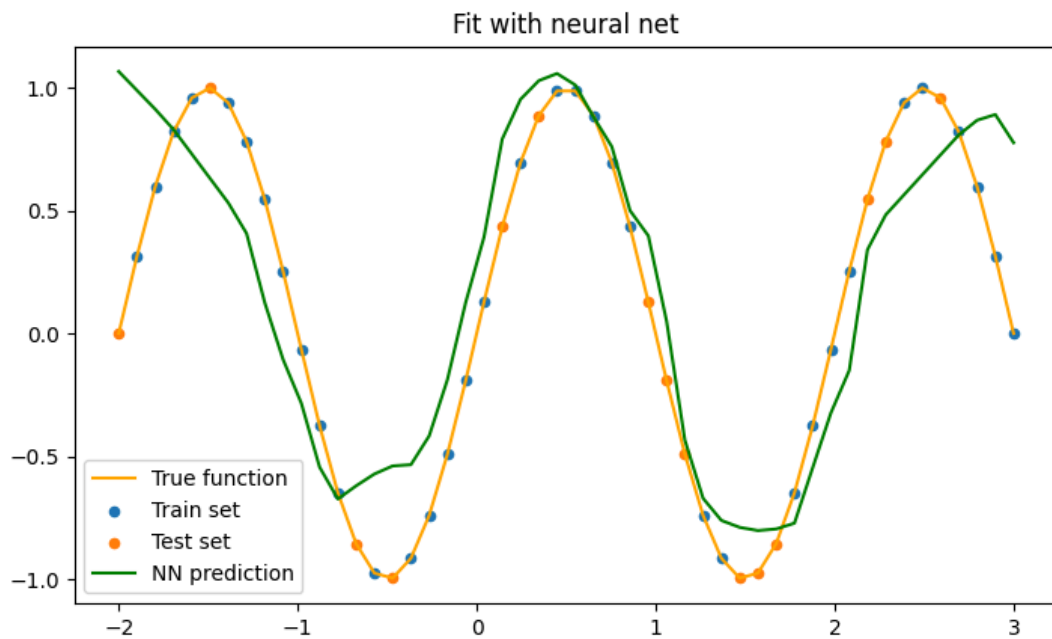


Figura 19 — Comparação entre a função alvo e a predição obtida pela rede neural.
- com probabilidade de dropout de 0.4 na primeira camada.

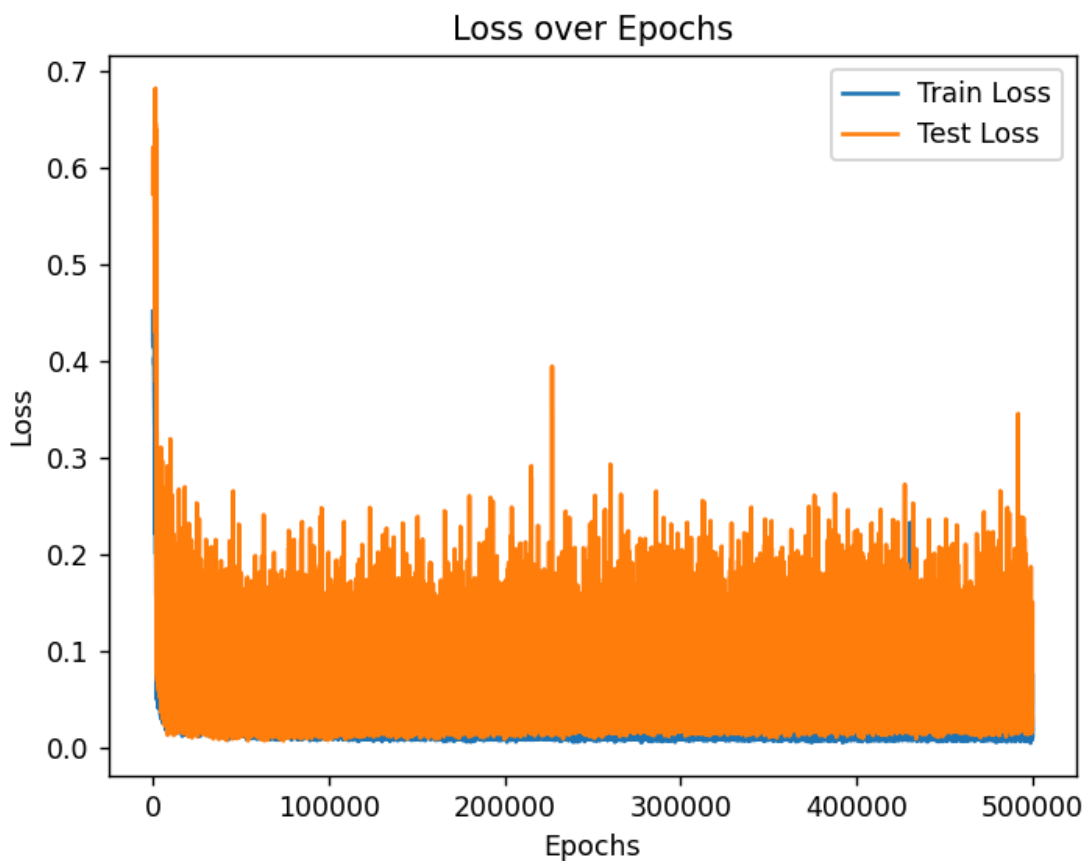


Figura 20 — Evolução da perda durante o treinamento - com probabilidade de dropout de 0.2 em ambas camadas.

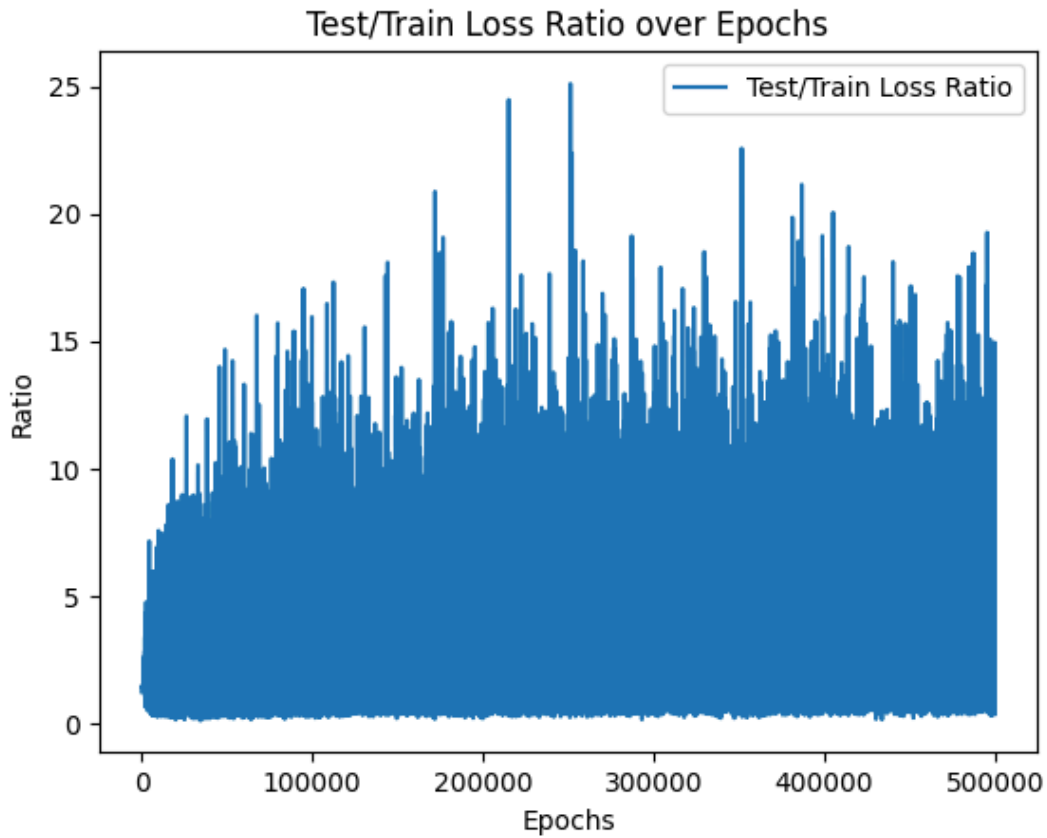


Figura 21 — Razão entre perdas de teste e treino ao longo do treinamento - com probabilidade de dropout de 0.2 em ambas camadas.

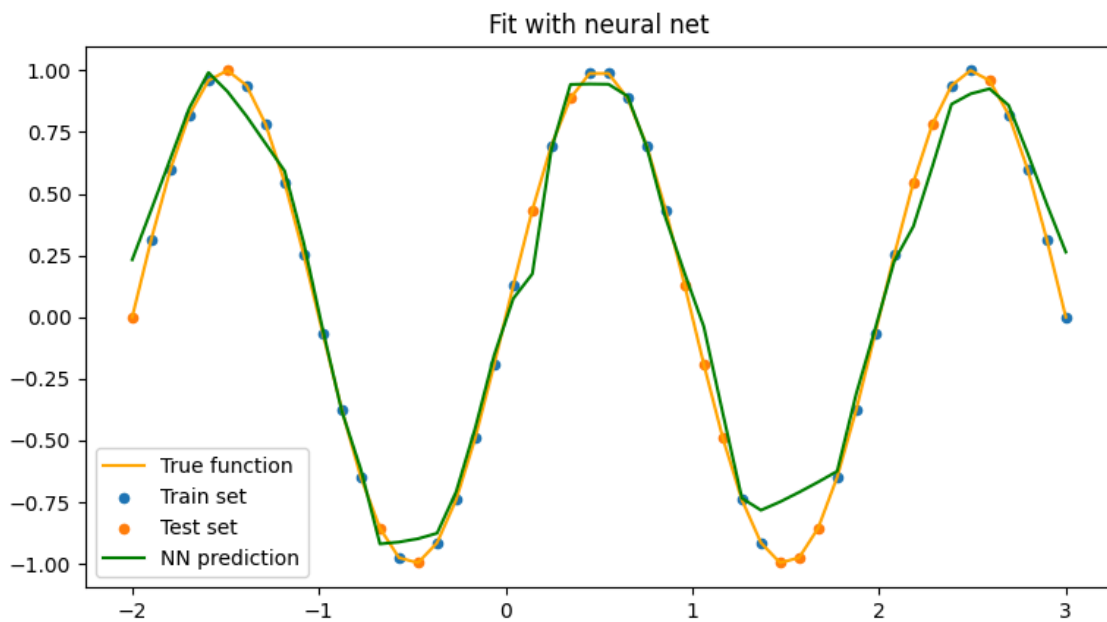


Figura 22 — Comparação entre a função alvo e a predição obtida pela rede neural. - com probabilidade de dropout de 0.2 em ambas camadas.

Durante o treinamento, o uso do *dropout* faz com que a rede aprenda de maneira perturbada, no qual diferentes subconjuntos de neurônios são removidos a cada iteração. Isso força o modelo a se adaptar constantemente a arquiteturas parciais. Como consequência pode-se verificar que o cálculo da perda tende a apresentar irregularidades.

4.2. Segunda Função

4.2.1. Resultados com Optimizador SGD (Sem Dropout)

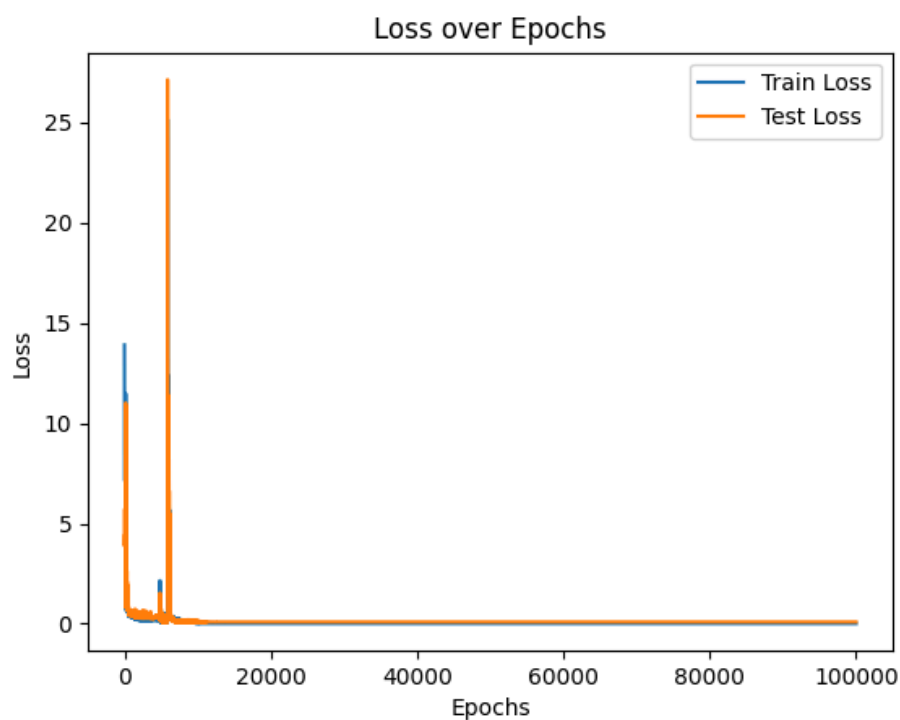


Figura 23 — Evolução da perda durante o treinamento, apresentando perda zero ao final do treinamento.

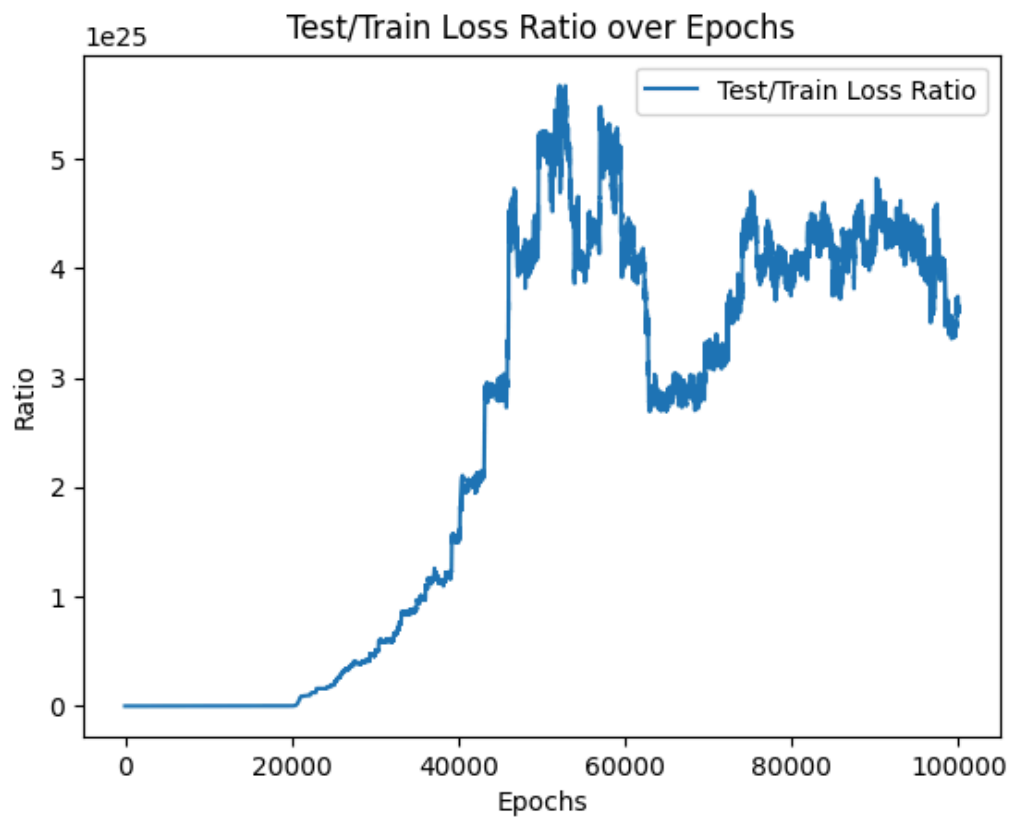


Figura 24 —Razão entre perdas de teste e treino ao longo do treinamento.

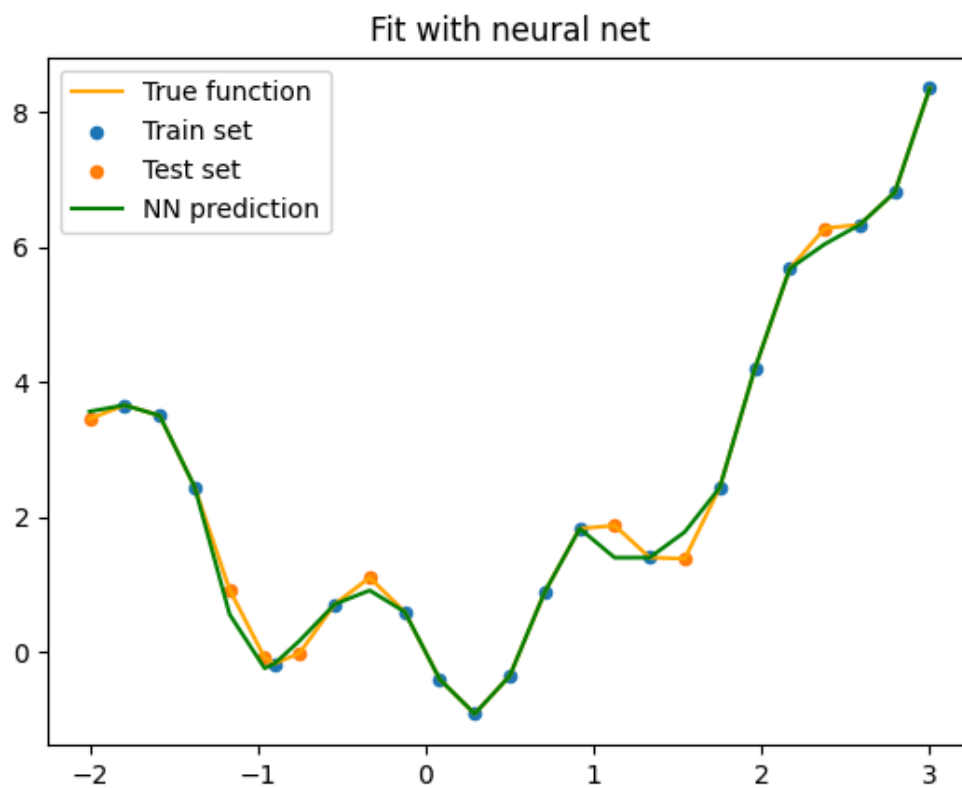


Figura 25 —Razão entre perdas de teste e treino ao longo do treinamento.

4.2.2. Resultados com taxa de Dropout de 0.2 na primeira camada

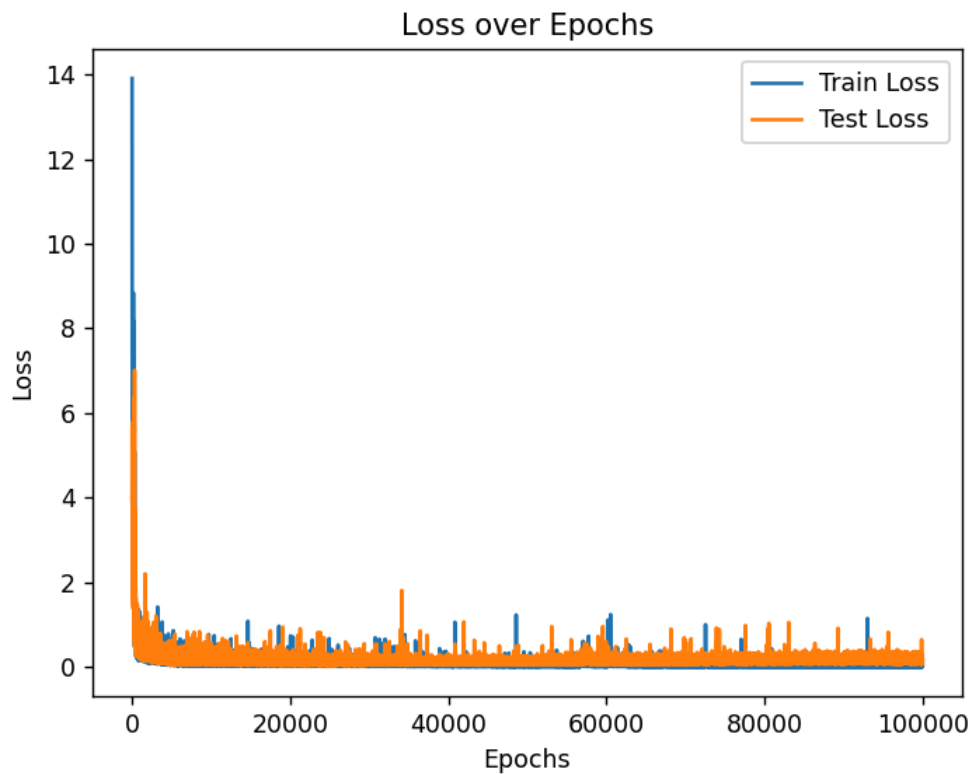


Figura 26 — Evolução da perda durante o treinamento.

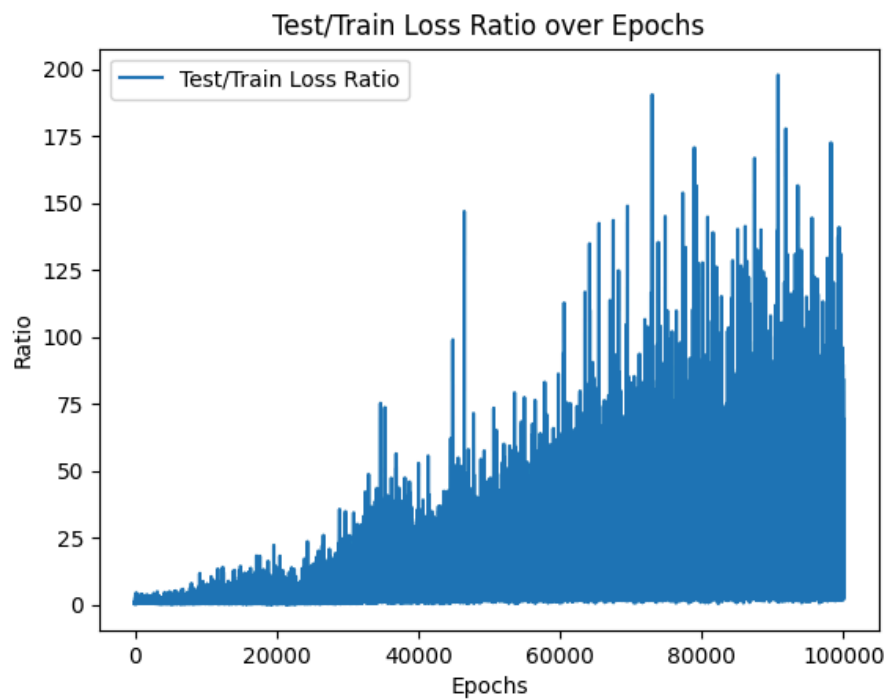


Figura 27 — Razão entre perdas de teste e treino ao longo do treinamento.

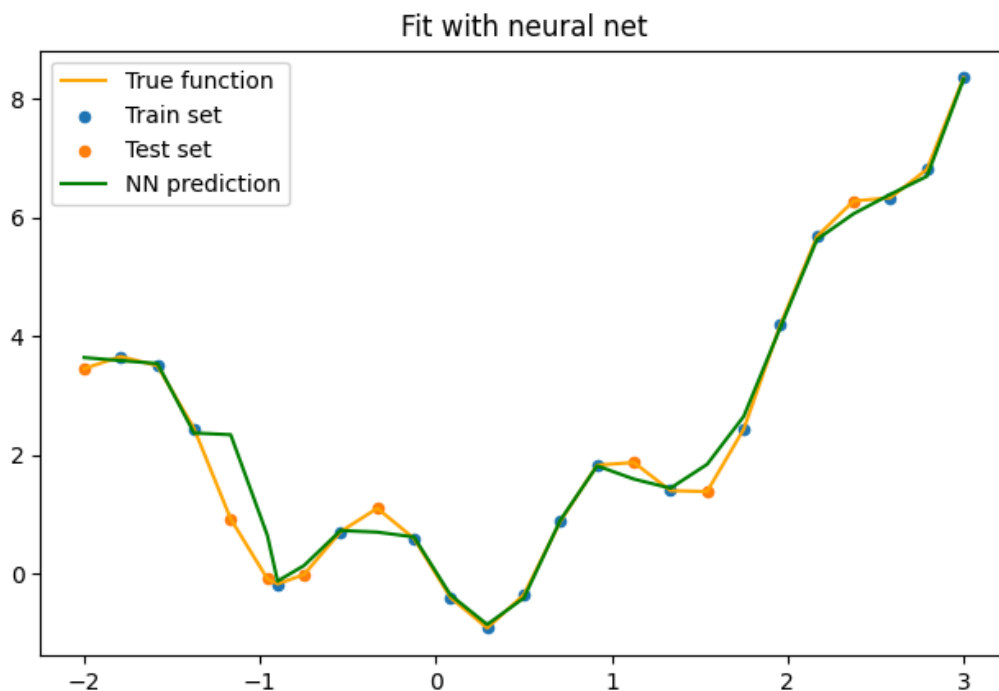


Figura 28 — Comparação entre a função alvo e a predição obtida pela rede neural.

4.2.3. Resultados com taxa de Dropout de 0.4 na primeira camada

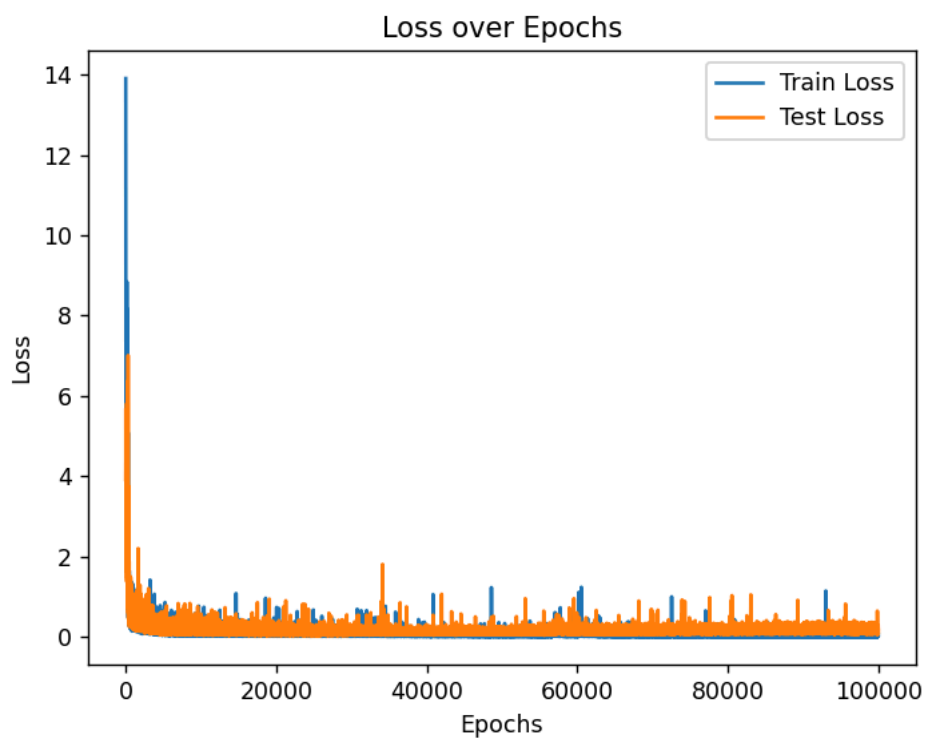


Figura 29 — Evolução da perda durante o treinamento.

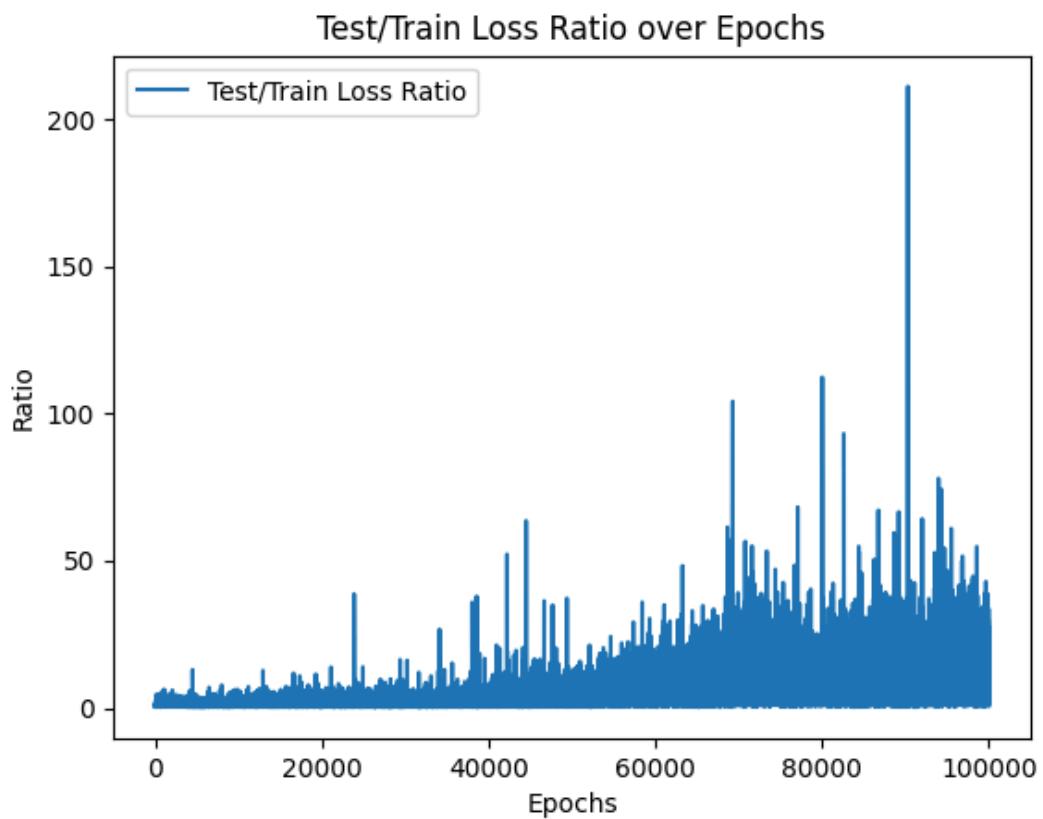


Figura 30 — Razão entre perdas de teste e treino ao longo do treinamento.

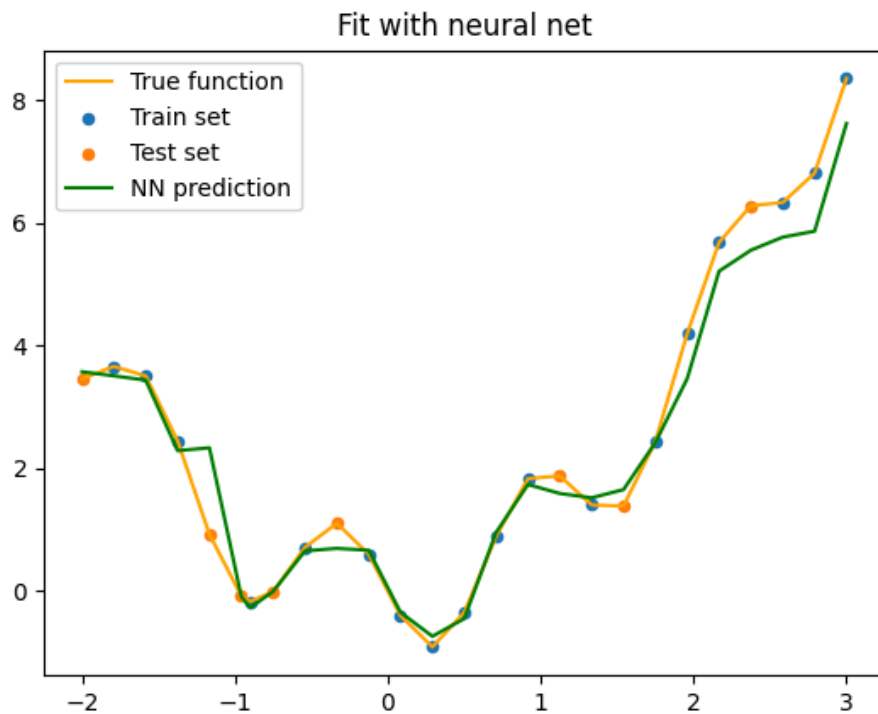


Figura 31 — Comparação entre a função alvo e a predição obtida pela rede neural.

4.2.4. Resultados com taxa de Dropout de 0.2 em todas as camadas

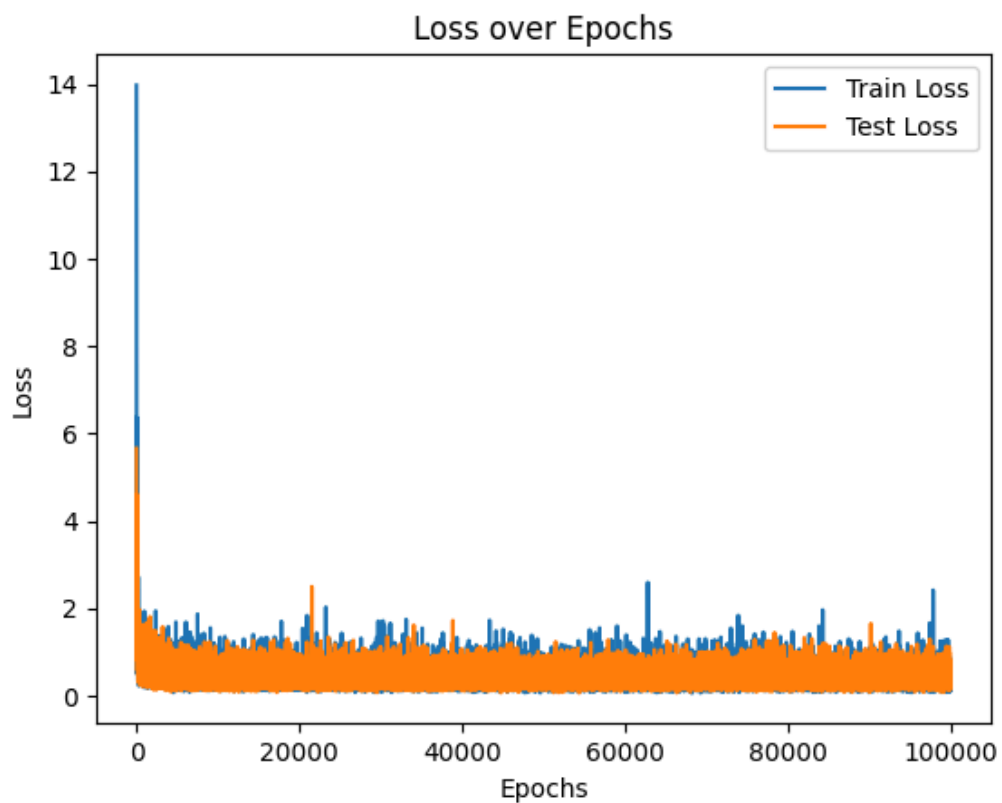


Figura 32 — Evolução da perda durante o treinamento.

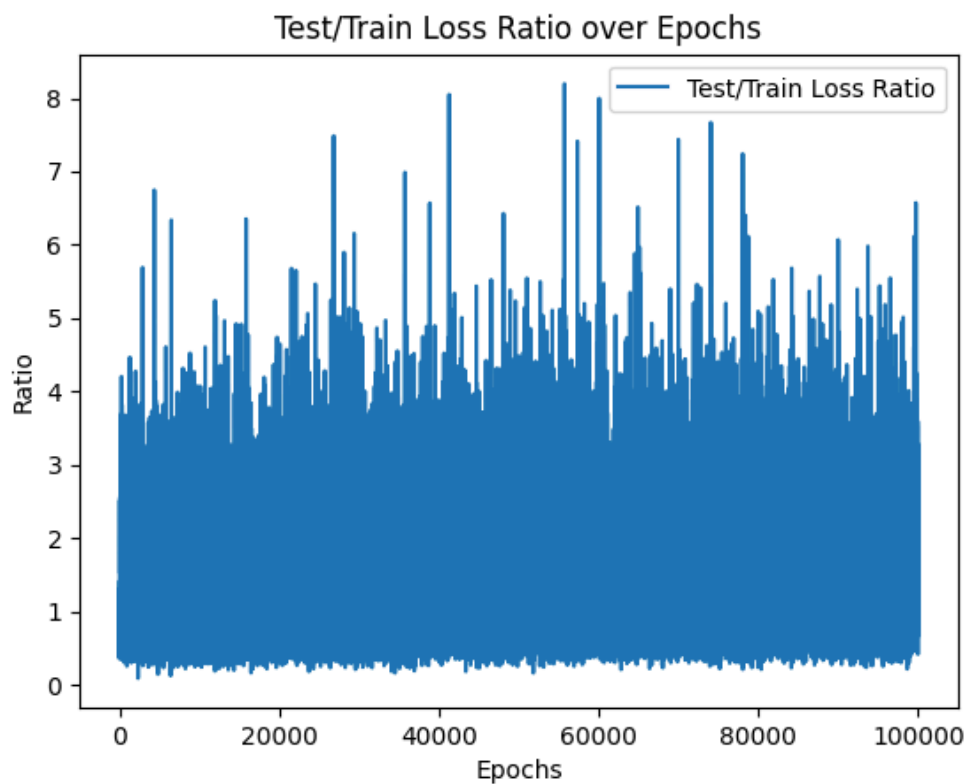


Figura 34 — Razão entre perdas de teste e treino ao longo do treinamento.

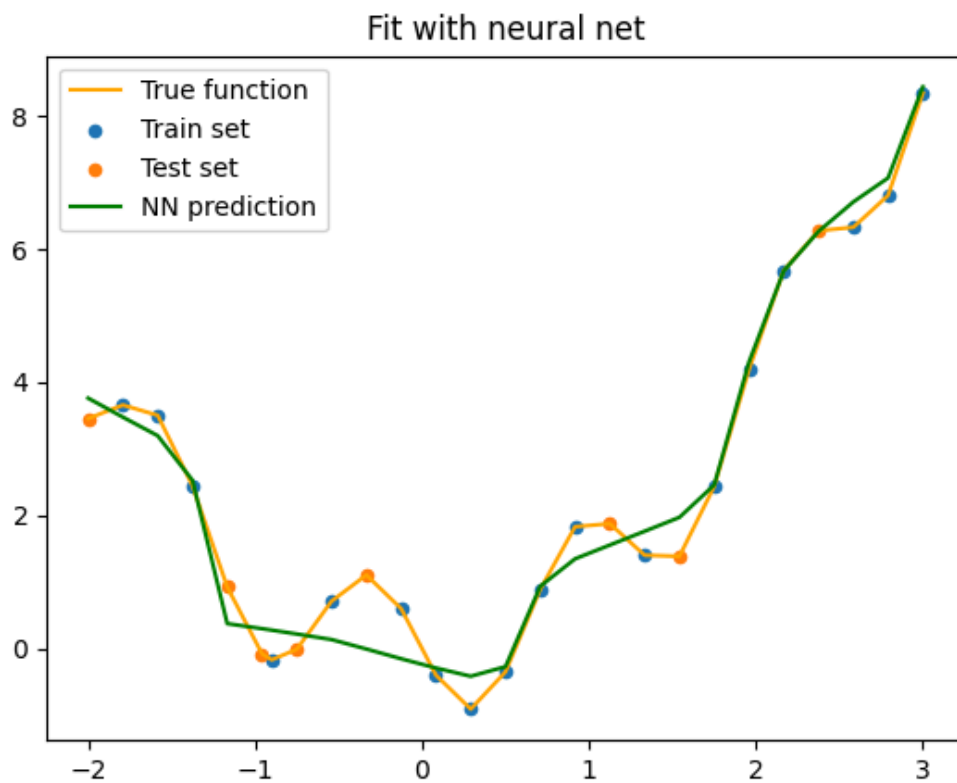


Figura 35 — Comparação entre a função alvo e a predição obtida pela rede neural.

4.2.5. Resultados com taxa de Dropout de 0.4 em todas as camadas

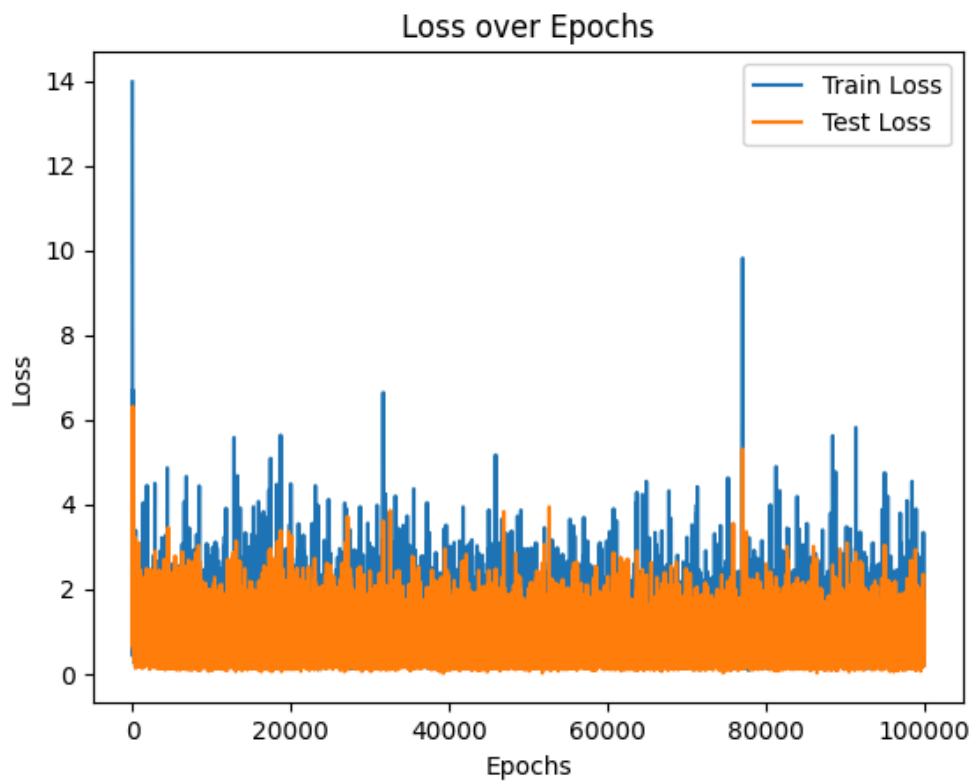


Figura 36 — Evolução da perda durante o treinamento.

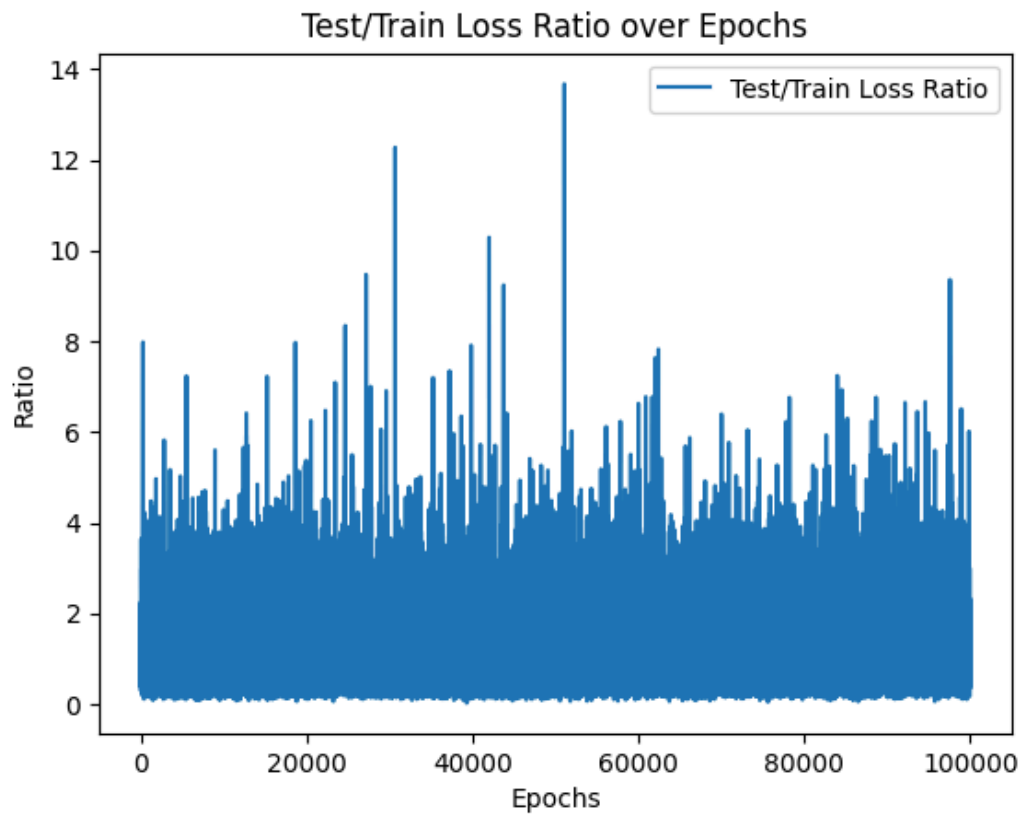


Figura 37 — Razão entre perdas de teste e treino ao longo do treinamento.

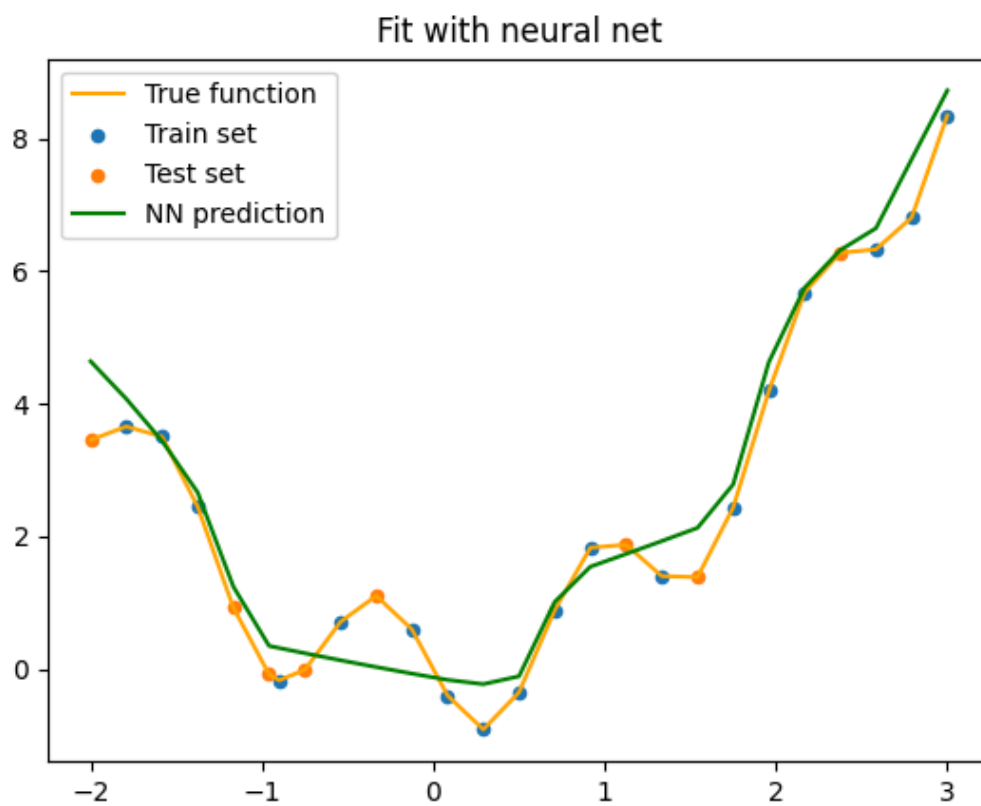


Figura 38 — Comparação entre a função alvo e a predição obtida pela rede neural.

4.2.6. Resultados com taxa de Dropout de 0.8 na primeira camada

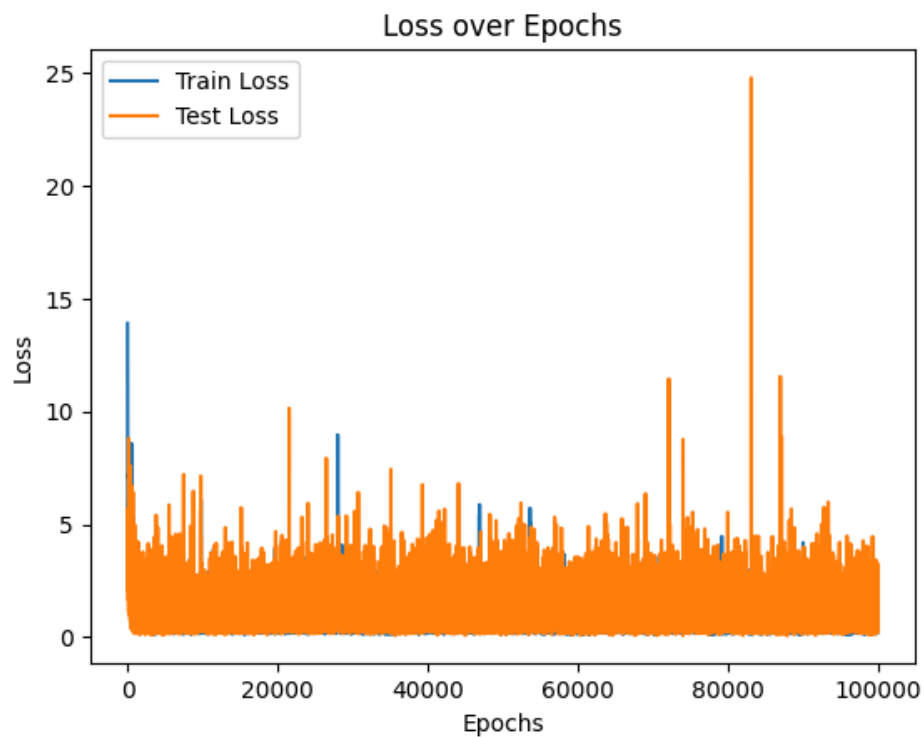


Figura 39 — Evolução da perda durante o treinamento.

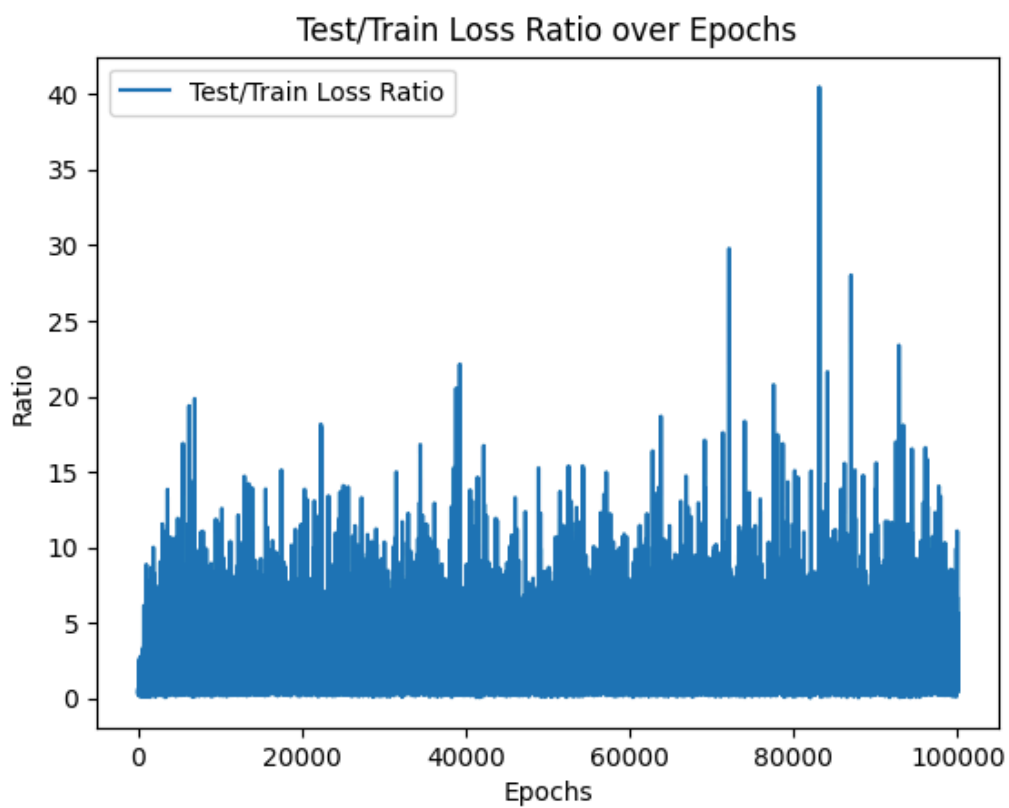


Figura 40 — Razão entre perdas de teste e treino ao longo do treinamento.

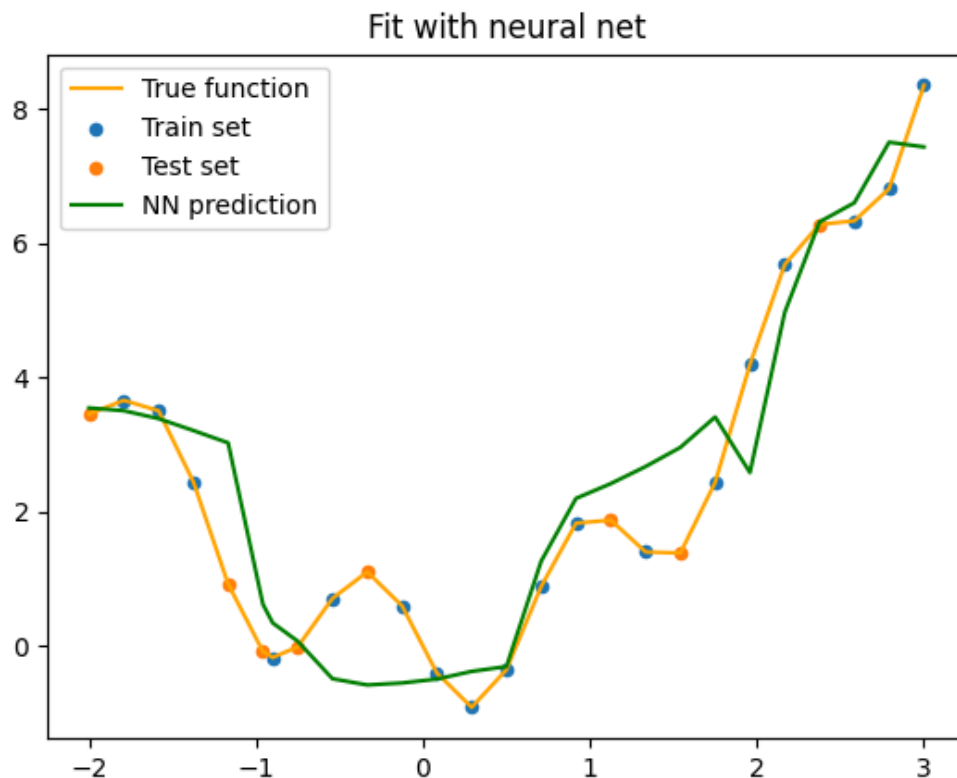


Figura 41 — Comparação entre a função alvo e a predição obtida pela rede neural.

4.2.7. Resultados com taxa de Dropout de 0.4 na última camada

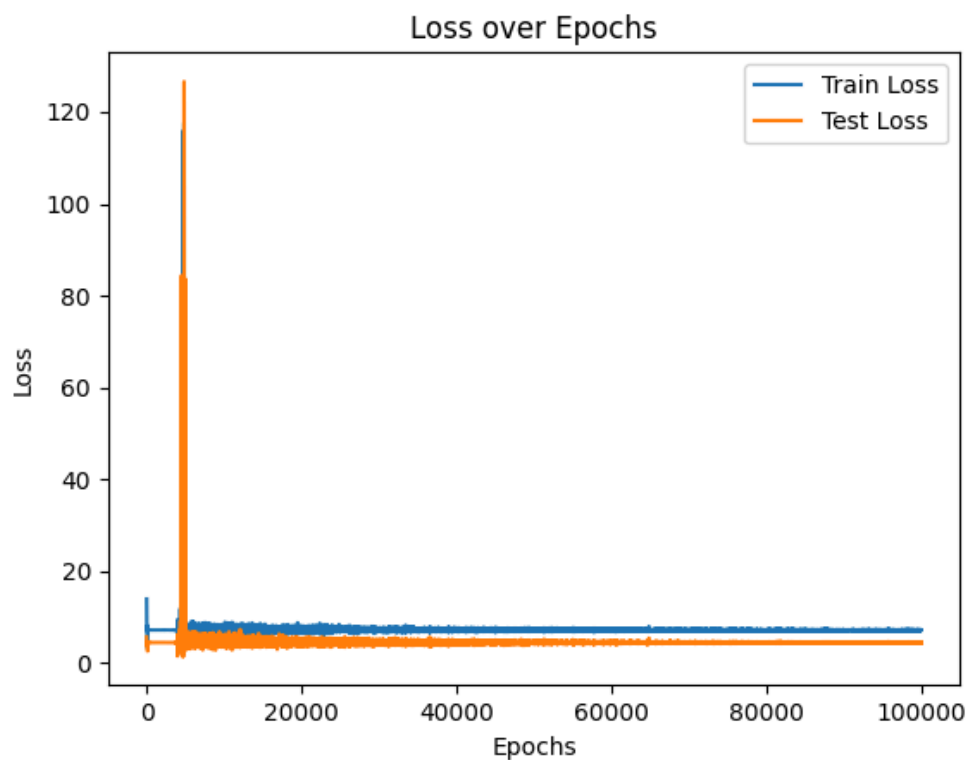


Figura 42 — Evolução da perda durante o treinamento.

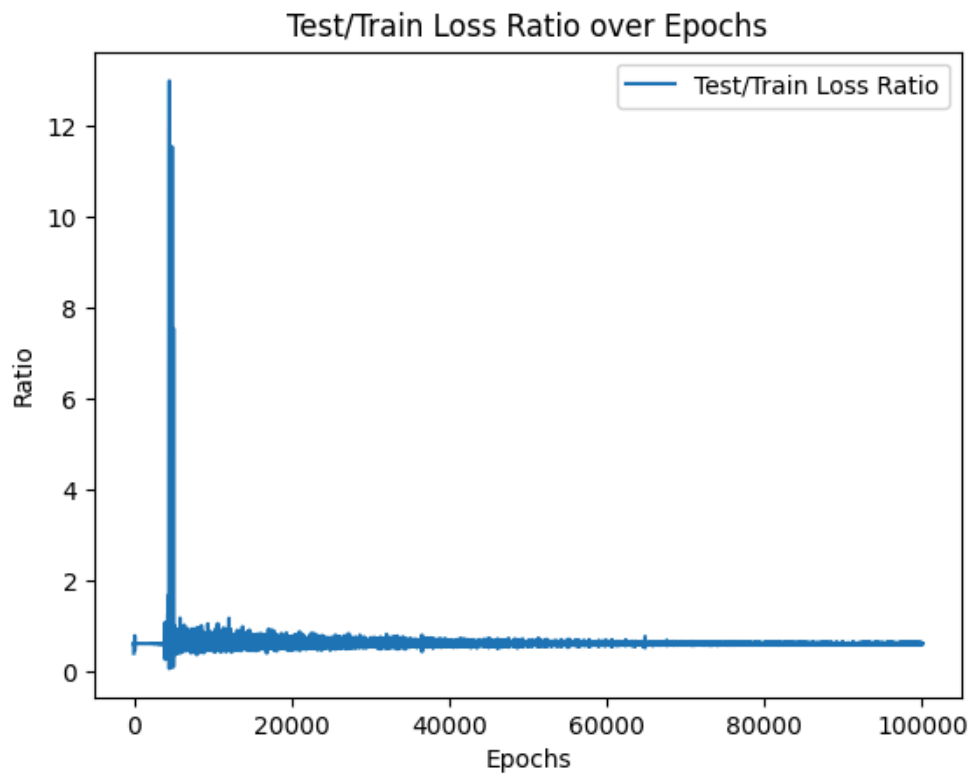


Figura 43 — Razão entre perdas de teste e treino ao longo do treinamento.

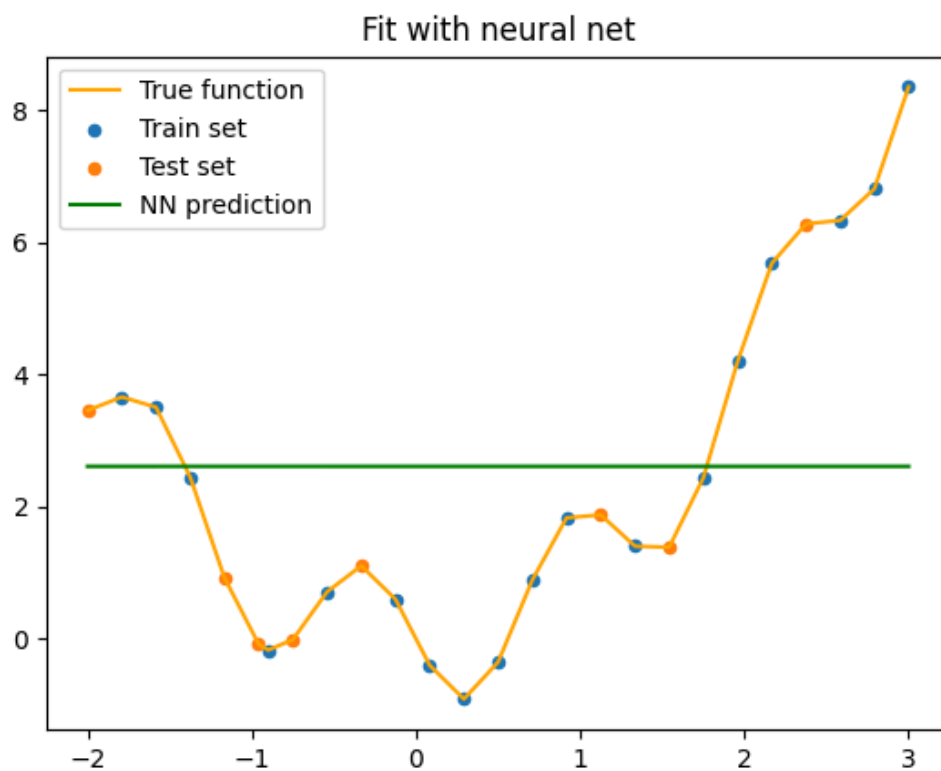


Figura 44 — Comparação entre a função alvo e a predição obtida pela rede neural.

5. Conclusões

O estudo teve como objetivo avaliar o impacto do uso do dropout em uma rede neural simples aplicada a funções determinísticas, especificamente uma função seno e uma função polinomial. Embora o *dropout* seja tradicionalmente utilizado como mecanismo de regularização em cenários com dados complexos ou de alta dimensionalidade, a investigação mostrou que, mesmo em problemas relativamente simples, ele foi capaz de atenuar o overfitting.

É importante destacar que, por se tratar de funções suaves e completamente determinísticas, o overfitting tende a gerar curvas visualmente “boas”, isto é, aproximações muito próximas da função-alvo, ainda que a rede esteja memorizando o conjunto de treinamento. Nesses casos, a redução do overfitting por meio do dropout não resulta necessariamente em um aspecto visual mais interessante. Ainda assim, é perceptível que o mecanismo conseguiu agir como regularizador.

No entanto, observou-se que, em diversos testes, a introdução do *dropout* adicionou mais incerteza do que vantagem, e seus benefícios não se manifestaram de forma consistente. O caso em que a técnica foi aplicada exclusivamente na última camada da rede, demonstrou de maneira clara um risco do seu uso. Nessa configuração, o método não apenas deixou de reduzir o overfitting como também aumentou significativamente o erro de predição. A rede tornou-se incapaz de representar a função-alvo, produzindo uma saída quase constante para todos os pontos. Esse resultado mostra que o uso inadequado do *dropout* pode desestabilizar o aprendizado ao invés de melhorá-lo. Assim, longe de atuar como regularização útil, o *dropout* acabou comprometendo o modelo.

De maneira geral, conclui-se que o *dropout* deve ser utilizado com cuidado e não como estratégia padrão de regularização. Dada a sua natureza estocástica e seu potencial para atrapalhar as predições, seu uso isolado pode apresentar pouca eficácia nos cenários analisados. Uma abordagem mais adequada parece envolver a combinação do *dropout* com outros métodos de regularização e otimização, permitindo que o *dropout* atue apenas como componente complementar dentro de um conjunto de técnicas para controle de sobreajuste.

Referências

- [1] KINSLEY, Harrison; KUKIEŁA, Daniel. Neural Networks from Scratch in Python. 1. ed. [S.l.]: [s.n.], 2020.
- [2] GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. Deep Learning. Cambridge (MA): MIT Press, 2016. ISBN 978-0262035613.
- [3] SCHMIDHUBER, Jürgen. Deep learning in neural networks: An overview. Neural Networks, v. 61, p. 85–117, 2015. doi:10.1016/j.neunet.2014.09.003
- [4] PIRES, Paulo Botelho; SANTOS, José Duarte; PEREIRA, Inês Veiga. Artificial Neural Networks: History and State of the Art. In: KHOSROW-POUR, Mehdi (Org.). Encyclopedia of Information Science and Technology, Sixth Edition. Hershey (PA): IGI Global, 2025. p. 1–25. doi: 10.4018/978-1-6684-7366-5.ch037.